APPLE'S ON-DEVICE FOUNDATION MODEL, REBUILT: A FROM-WEIGHTS RECONSTRUCTION OF `AFMPLUS-V11.2-IFP` AND A PORTABLE LLAMA.CPP RUNTIME

YU YAO-HSING

Abstract. Apple's on-device foundation models are distributed as proprietary assets compiled for the Neural Engine, which makes their structure and inference behaviour difficult to study outside Apple's runtime. We reconstruct the language model that Apple Intelligence installs on macOS 27, `afmplus-v11.2-ifp`, from its shipped assets, and implement a portable inference backend for it in llama.cpp. The recovered model has 56 transformer layers and 20.30 billion stored parameters. It combines dense layers, sparse feed-forward experts selected by a router (Instruction-Following Pruning), and key/value reuse across its last 21 layers. We decode the quantised weights and the tokenizer, identify a non-standard residual formulation in which both residual additions carry normalised values, and recover the prompt format and the expert-selection schedule from Apple's inference frameworks. No parameter is trained or fitted.

We evaluate against Apple's own model, queried through the public Foundation Models API at temperature zero. Under teacher forcing, the reconstruction's greedy next-token prediction matches Apple's at 95.66% of positions on a held-out set of 100 prompts (2,900 reference tokens) that played no part in development, and at 95.19% on the 230 prompts (6,572 tokens) used during development. This is a measure of token-level agreement, not of full-response identity: 51 of the 100 held-out answers are reproduced character for character. Exact numerical equivalence with Apple's compiled Neural Engine program is not achieved; the remaining disagreements are concentrated at near-ties between the two most likely tokens.

We also decode the companion components—the feature adapters, the speculative-decoding draft model, the image encoder, and the 300M model with its safety guard—and validate each to the extent that a reference exists. The released converter and runtime patch rebuild the models from assets already present on the user's Mac; no Apple weights are redistributed.

Contents

1. Introduction 2
2. Identification of the model 4
3. Model assets 5
4. Architecture 6
4.1. Dimensions 7
4.2. The layer: a post-normalised "x/y" residual 8
4.3. Attention 8
4.4. The Instruction-Following-Pruned feed-forward network 9
5. How the weights are stored 9
6. The norm-gain table and its alignment 10
7. Validation instruments and their calibration 11
8. Apple's runtime: the prompt template and the expert schedule 13
8.1. The prompt template 13
8.2. The expert schedule 14
9. The llama.cpp implementation 14
10. The converter 16

Date: November 14, 2026.

11. Evaluation methodology 17
12. Fidelity 18
13. The remaining gap 21
14. Apple Intelligence feature adapters 23
15. The speculative-decoding draft model 24
15.1. What the asset encodes 25
15.2. What Apple's runtime appears to do 25
15.3. In llama.cpp 26
16. The image encoder 27
17. The 300M base model 30
18. Limitations 33
19. Related work 33
20. Responsible use 34
21. Conclusion 34
Appendix A. Negative results 35
Appendix B. Experiments on the uncorrected configuration 36
Appendix C. GGUF layout 37
References 37

1. Introduction

Modern on-device language models are often available to users only through a platform API, even when their parameter data reside on the user's own computer. Reproducing such a model requires more than extracting tensors: the execution graph, the compressed storage layout, the tokenizer, the prompt convention and the runtime's policies each affect its output, and for a model compiled for a proprietary accelerator none of them is documented. Apple's on-device foundation model is a case in point. Its weights ship inside a compiled Apple Neural Engine (ANE) program whose format is undocumented; its feed-forward layers are sparse, with the active subset chosen at run time by code that is not part of the model files; and the kernel refuses to run the program on behalf of third-party processes (Appendix A). Published descriptions of Apple's models [1, 2] give the design principles but not the shipped model's dimensions, storage format or scheduling rules.

This study asks how much of one locally installed Apple foundation model can be reconstructed from its model assets and from observable platform behaviour, without retraining the model and without redistributing its parameters.

What "reconstruction" means here. We recover the model's stored tensors, its principal computation graph and the runtime policies that can be observed from outside (the prompt template and the expert-selection schedule), and we implement a portable approximation of its inference behaviour. We do not claim numerically identical execution: the exact arithmetic of Apple's compiled Neural Engine program is not reproduced, and Section 13 quantifies what that leaves open.

Contributions.

(1) **A structural reconstruction** of the installed `afmplus-v11.2-ifp` language model and its weight encodings: 56 layers, a post-normalised residual with four learned gains per layer, grouped-query attention with an int8 key/value cache shared by the last 21 layers, and a sparse feed-forward network with 23 always-on and 23 routed experts per layer (Sections 4–6).

(2) **A converter and a llama.cpp implementation** of the recovered architecture. The 2-bit weights map onto an existing GGUF quantisation type without loss, and the runtime runs on macOS, Linux and Windows (Sections 9 and 10).

(3) **A reconstruction of prompt rendering and expert routing** from Apple's inference frameworks and runtime logs (Section 8).
(4) **An evaluation against Apple's own model**, with ablations that isolate the implementation details that matter and a held-out test that was not used during development (Sections 11 and 12).
(5) **An analysis of the companion components**—feature adapters, draft model, image encoder, 300M model and safety guard (Sections 14–17)—and of the remaining numerical discrepancy (Section 13).

Sources of evidence. The claims in this paper rest on evidence of four kinds, and we try to keep them apart. *Asset bytes* are directly observed: file contents, tensor values, constants. *Program interpretation* is inference from compiled artefacts—the Neural Engine task stream, register values, disassembled framework code—about what the model computes. *Controlled comparison* checks an interpretation against a reference we can run: Apple's model through the public API, Apple's own compiler applied to known weights, Apple's image encoder executed directly, or values in Apple's runtime log. A *hypothesis* is consistent with the evidence but has not been tested directly. Table 1 classifies the principal claims. We use "observed" for asset bytes and log values, "validated" for controlled comparisons, and "indicates" or "is consistent with" for indirect evidence.

Claim	Evidence	Basis
Tensor values, palettes, row scales	observed; validated	file bytes; layout confirmed by compiling known weights with Apple's compiler
Dimensions, head counts, layer plan	interpretation	convolution shapes and task addressing in the compiled program
Post-normalised residual wiring	interpretation; validated	memory-transfer pattern of the program; a standard residual scores 0% against Apple
Alignment of the norm-gain table	observed; validated	64-byte alignment; coupling-sign agreement 98–99.9%; the model functions only when aligned
Prompt template	interpretation; validated	disassembly; rendered length equals Apple's token count on 5 of 5 transcripts
Expert-selection schedule	interpretation; indirect	disassembly and runtime log; counts of changed experts correlate with Apple's ($r = 0.84$, 0.93), identities are not observable
Layer-0 input (no multiplier; bias for generated tokens only)	interpretation; validated	two procedure families in the program; agreement with Apple
Neural Engine SiLU table	observed; validated	table bytes; tabulated on the hardware for all fp16 inputs
Translation and urgency adapters	validated	Apple's translations and logged verdicts
Remaining 24 feature adapters	observed only	same file format and decode; no reference available
Draft model weights and graph	interpretation; indirect	task stream; accept/reject counters in Apple's log
Tree-shaped speculation	hypothesis, supported	metadata flag and counters; chain and other tree sizes fit worse
Image encoder	validated	Apple's encoder executed directly on our inputs
Host image preprocessing	indirect	behaviour of the 3B model on images
300M base model	behavioural only	perplexity, induction, coherent text; Apple exposes no reference
Safety guard	behavioural comparison	guardrail decisions of the public API on 100 prompts
Cause of the residual disagreement	hypothesis	Section 13

Table 1. Evidence ledger for the principal claims.

Terminology. Table 2 defines the terms and abbreviations used throughout.

ANE	Apple Neural Engine, the accelerator for which the model is compiled
`.hwx`	a compiled ANE program: procedures, weight segments and constants
ODIE, `.odix`	Apple's orchestration program format; here it holds the embeddings and position tables
MPSGraph	Apple's graph framework; the model ships as an MPSGraph package that delegates to the ANE
tamm	the name of Apple's modelling library as it appears in the graphs
IFP	Instruction-Following Pruning [2]: a router selects which feed-forward parameters are active
expert	one 256-wide SwiGLU block of a sparse layer's feed-forward network
budget (ifp1, ifp2, ifp3)	the number of experts active per sparse layer: 5, 14 or 23 always-on plus as many routed
KV reuse	layers 35–55 compute queries only and attend to the keys and values of layer 34
GGUF; Q2_K, Q4_0	llama.cpp's model file format [7] and two of its quantisation types
LoRA	low-rank adapter [12]; Apple ships its feature adapters in this form
3B, 300M, 9M	Apple's size-class names for the main model, the small model and the draft model. They are names, not parameter counts; Table 8 gives the counts
reference (oracle)	Apple's own model, queried through the public Foundation Models API at temperature zero
teacher-forced top-1 agreement	Apple's answer is fed in token by token; the share of positions at which our most likely next token is Apple's
free-running identity	our own greedy answer equals Apple's character for character

Table 2. Terminology.

Summary of results. Table 3 summarises the main model and Table 4 the companion components, with the kind of validation available for each. All measurements were taken on one Apple M4 Mac running macOS 27.0.

Model	`afmplus-v11.2-ifp`, on-device, sparse (IFP)
Parameters	20.30 B in the converted model; about 1.2, 2.1 or 3.0 B used per token, by budget (Table 8)
Layers	56: 12 dense, 23 sparse with their own K/V, 21 sparse reusing layer 34's K/V
Width	$d = 1536$; 16 query and 8 key/value heads of 128; dense FFN 3072; experts 256 wide
Weights	2-bit palette × fp16 row scale; int4 embedding
GGUF	6.31 GiB; Q2_K and Q4_0, both lossless encodings of Apple's values
Tokenizer	reproduces Apple's token count on 81 of 81 texts
Runtime rules	prompt template and expert schedule recovered from Apple's frameworks
Held-out agreement	95.66% teacher-forced top-1 (100 prompts, 2,900 tokens; 95% interval 94.8–96.5%)
Development sets	96.98% (40 prompts, 1,127 tokens); 95.19% (230 prompts, 6,572 tokens)
Identical answers	51 of 100 held-out; 17 of 40 development

Table 3. The main model. Agreement figures are against Apple's model at temperature zero (Section 11).

Code: `https://github.com/yu314-coder/afm-ifp-teardown` (llama.cpp patch, converter, evaluation scripts and prompt sets, build workflow). No Apple model data is published; the converter rebuilds the GGUF from the assets on the user's own machine.

2. Identification of the model

Apple's third-generation foundation models (announced June 2026) comprise two on-device language models and several Private Cloud Compute models (Table 5). The macOS 27 SDK exposes the on-device model as `SystemLanguageModel` and the server models through

Component	Result	Validation
Feature adapters	26 rank-48 LoRAs decoded; translation 98.7% of 3,424 tokens, urgency 28 of 30 verdicts	2 checked against Apple; 24 decoded only
Draft model	12 layers, $d = 256$, 9.4 M; matches 99.2% of the clear accept/reject decisions in Apple's log	indirect (runtime log)
Image encoder	24-layer windowed ViT with 3×3 merger, 358 M; cosine 0.98–0.99 per token; 6 of 13 captions identical	checked against Apple's encoder
300M base model	24 layers, $d = 1024$; perplexity 21–31 with its default adapter	behavioural only
Safety guard	300M adapter; predicts the public API's guardrail decision on 95 of 100 prompts	behavioural comparison
300M image encoder	same structure as the 3B's; cosine 0.990–0.999 per token on photos	checked against Apple's encoder

Table 4. Companion components (Sections 14–17).

`PrivateCloudComputeLanguageModel`. The assets on the device contain exactly one foundation language backbone, `afmplus-v11.2-ifp`; the server models are present only as adapter metadata (`afmplus-v8.0-150b`, `afmplus-v9.0-3b-pcc`).

Name	Where	Size	Architecture	On this Mac?
AFM 3 Core	device	3 B	dense	no separate asset found
AFM 3 Core Advanced	device	**20 B**, 1–3 B active	**sparse, IFP**	**yes (identified)**
AFM 3 Cloud	PCC	n/a	n/a	metadata only
AFM 3 Cloud Pro	PCC (GPUs)	n/a	n/a	no

Table 5. Apple's 2026 lineup as described in public WWDC 2026 coverage (secondary sources; Apple's own documentation does not list sizes). The last column is ours.

Apple does not publish a mapping from asset identifiers to product names, so the name is an identification, not a disclosed fact. Based on three observations we identify the local asset with the on-device tier described as *AFM 3 Core Advanced*: the converted model holds 20.30 B parameters; its feed-forward network is Instruction-Following-Pruned; and the Foundation Models API ships three compute budgets (`fm_api_generic_1b/2b/3b`) that select 5, 14 or 23 routed experts, which correspond to about 1.2, 2.1 and 3.0 B parameters used per token (Table 8). It is not a "Pro" model: by the same public descriptions the Pro model runs only on Apple's servers, and no such asset is present on the device. In the rest of the paper we refer to the model by its asset version, `afmplus-v11.2-ifp`, or by Apple's size-class name "3B".

3. Model assets

Apple Intelligence downloads its models as MobileAssets under `/System/Library/AssetsV2/com_apple_MobileAsset_UAF_FM_GenerativeModels`. The model files sit in an APFS cryptex that the system grafts into the asset's `.AssetData` directory.

File	What it holds
`binary_0.hwx` (694 MB) in `main-h16g-delegates/MPSGraph/`	The compiled ANE program: 108 procedures (prompt / extend / dense-only / sparse-only / readout, per sequence length and IFP budget). Segment `__KERN_3` holds every attention and dense-FFN weight; `__TEXT,__const` holds the norm gains; the task stream holds the int8 K/V calibration.
`main-h16g.odix` (263 MB)	The orchestration program (ODIE IR). Holds the int4 token embedding (text, audio) and its per-token fp16 scales, and the RoPE tables.
`ifp/ifp_rasterized_weights.bin` (4.9 GB)	All 44×380 experts: 2-bit codes, gate/up and down row scales.
`ifp/project_experts.mlasset`	The IFP router network (fp16).
`ifp/config.json`	IFP budgets: ifp1/2/3 = 5/14/23 active experts; `expert_selection_frequency` = 32.
`tokenizer` (separate asset)	328 192-entry SentencePiece-BPE [14] vocabulary in a flat Apple format (262 144 text tokens + audio/voice tokens).

Table 6. Files used by the reconstruction.

Reading Apple's graphs. A private method on `MPSGraphExecutable`, `-getIR`, returns a loaded package's graph as MLIR text. On AFM it yields only placement operations and 108 calls into the ANE program, but with exact entry signatures (for example, the two 64×128 RoPE tables arrive as runtime inputs). On Xcode's code-completion model (CodeLM v3, a GPU model built with the same modelling library, *tamm*) it yields the complete source-level graph. CodeLM serves as our healthy reference model (Section 7).

4. Architecture

Figure 1 shows the model as a whole. The subsections that follow give its dimensions, the structure of a layer, the attention mechanism and the sparse feed-forward network.

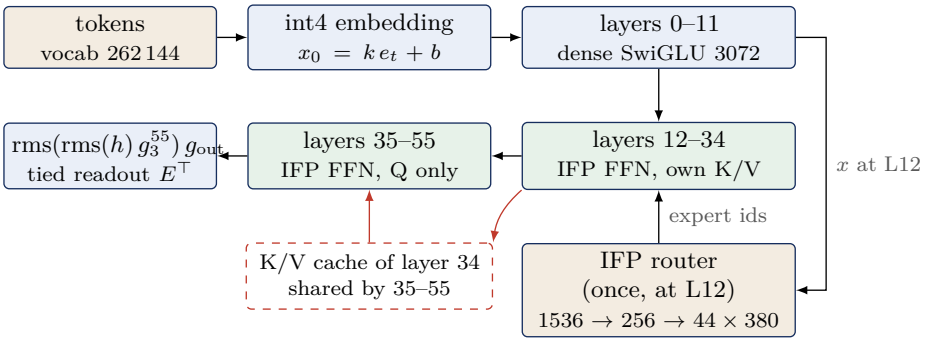


Figure 1. The whole model. Shapes are read from the task stream of the compiled program; the routing step and its timing come from the runtime analysis of Section 8; the residual arrangement is validated behaviourally. The router reads the normalised input of layer 12 and emits routing logits for all 44 sparse layers at once. Layers 35–55 compute queries only and attend to the K/V cache written by layer 34.

Quantity	Value	Evidence
layers	56 (12 dense, 23 sparse own-KV, 21 sparse KV-reuse)	chunk-set plan; task stream
hidden / head dim	1536 / 128	conv shapes
heads	16 query, 8 key/value; query head h uses KV head $\lfloor h/2 \rfloor$	attention task addressing; same-token probe
dense FFN	SwiGLU, width 3072 (layers 0–11)	conv shapes; SiLU lookup table on the gate
experts	index space 0–379 per sparse layer, width 256: 0–22 always on (23), 23–31 unused (9), 32–379 routable (348)	router mask; static kernels = experts 0–22
RoPE	NEOX half-split, $\theta = 5 \times 10^5$, 128 dims	odix cos table (row 0 = 1, halves equal)
norm ε	$\varepsilon = 1.0014 \times 10^{-5}$, added to the mean square inside the root	ANE register; identical in CodeLM
attention scale	$1/\sqrt{128}$	post-scale = $(1/s_K)/\sqrt{128}$ in the hardware; CodeLM `sdpa`
vocabulary	262 144 text tokens (the layout of the Gemma 3 family [15]), BOS 2, end-of-turn 106	tokenizer asset
context	8192	asset metadata

Table 7. Architecture parameters and where each was read.

4.1. **Dimensions.**

Parameter accounting. Three different numbers describe the model's size, and Table 8 separates them. The converted model holds 20.30 B parameters, 89% of them in routable experts. Only 23 of the 348 routable experts of a layer are active at a time, so the parameters *used* for a token are far fewer: about 3.0 B at the largest budget, which is where the size-class name "3B" comes from. The used counts are computed from the tensor shapes and the budgets; they are not measurements. Apple's expert file additionally reserves nine index slots per layer (23–31) that the router masks and no budget uses; they are not carried into the GGUF.

	stored	used per token		
Component		ifp1	ifp2	ifp3
Attention, 56 layers (21 without K/V)	0.462	0.462	0.462	0.462
Dense FFN, layers 0–11	0.170	0.170	0.170	0.170
Always-on experts, 44×23	1.194	0.260	0.727	1.194
Routable experts, 44×348	18.063	0.260	0.727	1.194
Router	0.004	0.004	0.004	0.004
Transformer total	19.893	1.155	2.089	3.023
Token embedding, $262{,}144 \times 1536$ (tied readout)	0.403	one row in; all rows at the readout		
Total	20.296			

Table 8. Parameter accounting, in billions. One expert has $3 \times 256 \times 1536 = 1.18$ M parameters. The budgets ifp1, ifp2 and ifp3 use the first 5, 14 or 23 always-on experts and as many routed ones. Norm gains and biases (0.4 M) are omitted.

4.2. **The layer: a post-normalised "x/y" residual.** In a standard pre-norm transformer the residual stream h is never normalised: each sublayer reads a normalised copy of h and adds its output to the unnormalised h. This model differs in two ways. The stream itself is renormalised before each sublayer, and each sublayer's output is normalised again before it is added. Both operands of both residual additions are therefore normalised vectors scaled by a learned gain.

Notation: h_{L-1} is the output of layer $L-1$; x_L is the normalised input of layer L's attention and y_L the normalised input of its feed-forward network; h'_L is the intermediate sum after attention. Each layer carries four learned RMSNorm gains $g_0, \ldots, g_3 \in \mathbb{R}^{1536}$, where g_0 scales the attention output, g_1 produces y, g_2 scales the feed-forward output, and g_3 produces the next layer's x:

$$
\begin{aligned}
x_L &= \mathrm{rms}(h_{L-1}) \odot g_3^{L-1} \qquad \text{(layer 0: } x_0 = k\, e_t + b, \text{ no norm)} \\
h'_L &= x_L + \mathrm{rms}\big(\mathrm{Attn}_L(x_L)\big) \odot g_0^L \\
y_L &= \mathrm{rms}(h'_L) \odot g_1^L \\
h_L &= y_L + \mathrm{rms}\big(\mathrm{FFN}_L(y_L)\big) \odot g_2^L
\end{aligned}
$$

with $\mathrm{rms}(v) = v/\sqrt{\mathrm{mean}(v^2) + \varepsilon}$ (RMSNorm [8]), gains used literally (no $1+g$), and a final output $\mathrm{rms}(\mathrm{rms}(h_{55}) \odot g_3^{55}) \odot g_{\text{out}}$ multiplied by the tied embedding. Read as one step: layer L normalises the previous output into x_L; attention reads x_L, and its normalised output is added to x_L itself, not to h_{L-1}; the sum is normalised into y_L; the feed-forward network reads y_L, and its normalised output is added to y_L. We call this the "x/y" residual after the two normalised streams.

The wiring is an interpretation of the ANE program's memory-transfer (DMA) pattern: the normalised attention input is written to scratch memory and read back by the attention residual add, and the feed-forward residual add reads the normalised feed-forward input. It is validated against Apple's outputs: with a standard residual and otherwise identical weights the model scores 0% top-1.

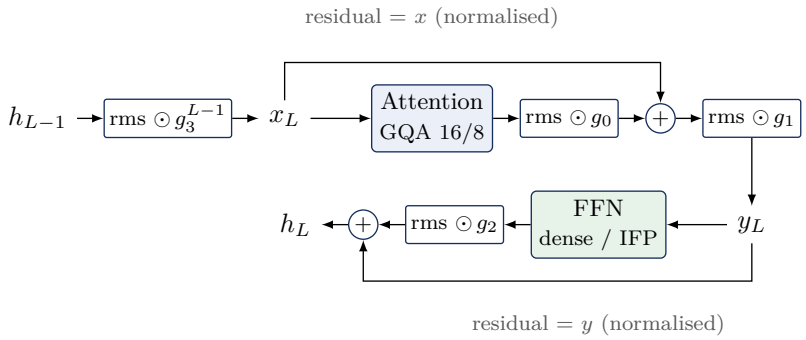


Figure 2. One layer. Both residual additions add normalised streams; g_0 and g_2 are zero-centred (about 30% negative entries), g_1 and g_3 small and positive. Zero-centred literal gains are normal in Apple's modelling library: a healthy Apple model (CodeLM) shows the same pattern.

4.3. **Attention.** Attention is standard grouped-query attention [10] with rotary position embeddings [9] and two Apple-specific details: the query/key normalisation sits after RoPE, and the key/value cache is int8 (Figure 3).

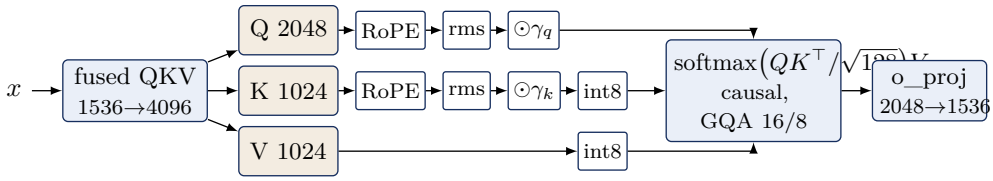


Figure 3. Attention as executed on the ANE. RoPE comes before the per-head RMS norm and its 128-entry gain (the same order appears in CodeLM's source graph). Keys and values are stored in an int8 cache with per-layer calibrated scales; layer 34 has no key gain. Layers 35–55 compute only Q and read layer 34's cache.

4.4. **The Instruction-Following-Pruned feed-forward network.** Sparse layers add two SwiGLU [11] blocks without any gating weights:

$$\mathrm{FFN}(y) = \underbrace{\sum_{e=0}^{22} \mathrm{SwiGLU}_e(y)}_{\text{always on}} + \underbrace{\sum_{e\in\mathcal{S}(y)} \mathrm{SwiGLU}_e(y)}_{|\mathcal{S}|=23 \text{ routed}}, \qquad \mathrm{SwiGLU}_e(y) = W_e^{\mathrm{down}}\big(\mathrm{SiLU}(W_e^{\mathrm{gate}}y)\odot W_e^{\mathrm{up}}y\big),$$

each expert being 256 wide. The ANE program runs the always-on experts from static kernels and the routed experts from a mutable kernel patched at runtime from `ifp_rasterized_weights.bin`. Both halves matter: the always-on experts alone reach 62.9% top-1, the routed experts alone 1.4%, both together 87.5% (measured with the single-turn template).

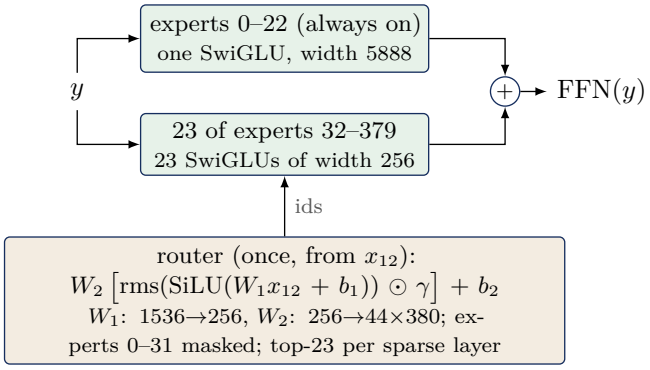


Figure 4. The IFP feed-forward network of a sparse layer and the router that feeds it. The sum is unweighted: the router only selects.

Expert indices. The expert index space of a sparse layer runs from 0 to 379. Experts 0–22 are always on. The router's mask excludes indices 0–31, so the 348 experts 32–379 are the eligible routed ones, of which the 23 with the largest router logits are active. Indices 23–31 are neither always on nor routable.

When experts are chosen. Apple's runtime does not route per token. It runs the dense layers over the whole prompt, takes the layer-12 input of the *last* prompt token, runs the router once, and uses that expert set for every prompt token and for generation; it re-picks from the current token only at absolute positions that are multiples of 32. Section 8 shows how this was read out of Apple's inference framework.

5. How the weights are stored

Exact GGUF encoding. llama.cpp's Q2_K stores, per 256 weights, sixteen 4-bit sub-block scales sc and mins m, a super-scale d and super-min $d_{\min}$, and evaluates $w = d\,sc\,c - d_{\min}\,m$. Choosing

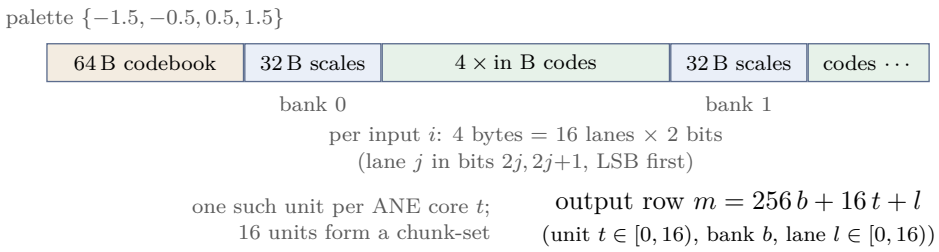


Figure 5. The 2-bit weight unit. A projection is split across the 16 ANE units bank-cyclically; a chunk with more banks holds the lower bank slots. The value of a weight is $w = s_m\,(c - 1.5)$ with $c \in \{0, 1, 2, 3\}$ and s_m the fp16 row scale. Experts in `ifp_rasterized_weights.bin` use the same codes with a simpler order: code $p \mapsto$ (row $16\lfloor p/(16\,\text{in})\rfloor + p \bmod 16$, input $\lfloor p/16\rfloor \bmod \text{in}$).

$sc = 2$, $m = 3$ and $d = d_{\min} = s/2$ gives

$$w = \tfrac{s}{2} \cdot 2c - \tfrac{s}{2} \cdot 3 = s\,(c - 1.5),$$

where $c \in \{0, 1, 2, 3\}$ is the 2-bit code and s the fp16 row scale. Halving an fp16 number is exact (barring underflow, which does not occur in these scales), so d and $d_{\min}$ are representable. The int4 embedding ($w = s_t\,q$, $q \in [-7, 7]$, per-token fp16 scale s_t) maps exactly onto Q4_0 with $d = s_t$ and nibble $q + 8$.

What "lossless" means here. The claim concerns decoded real values: the number a GGUF reader obtains for a weight equals the number Apple's format encodes, $s\,(c - 1.5)$ or $s_t\,q$, exactly. It rests on the algebra above, and on a check in which 55 million weights were dequantised with gguf-py and compared with the values decoded directly from Apple's files, with a maximum absolute difference of 0. That check was a sample, not an exhaustive comparison of all 20.3 billion weights. A separate statement, that the converter's output is byte-identical to the file used for our measurements (Section 10), concerns reproducibility of the conversion and is a different test. Neither says anything about arithmetic at inference time, where llama.cpp and the Neural Engine differ (Section 13).

Layout facts and how they were ground-truthed. We compiled known weights through Apple's own ANE compiler (`mil_to_hwx -a h16`) and decoded the result: the output axis is the bank-cyclic stripe above, the input axis is natural, the codebook and row scales are as in Figure 5, and constants (gains, biases) are stored in natural order. A calibrated copy-circuit probe (Section 7) is consistent with the stripe, the head order and the V/O pairing: every alternative layout scores at chance.

6. The norm-gain table and its alignment

All gains live in one contiguous fp16 table inside `__TEXT,__const` of `binary_0.hwx`:

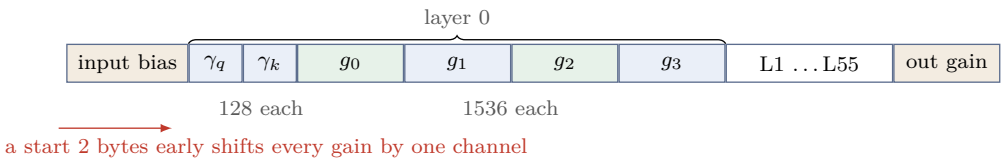


Figure 6. The gain table. Layers 0–33 carry γ_q and γ_k; layers 34–55 only γ_q. Every vector length is a multiple of 64 bytes, so a correct start is 64-byte aligned.

Finding 1. An initial parse located the table by statistical signatures and accepted a start two bytes early. Every RMSNorm therefore multiplied channel r by the gain of channel $r-1$. Because g_0 and g_2 are zero-centred, this flipped the sign of about a third of every residual write at every layer, and the assembled model predicted nothing (0% top-1, no induction).

How it was found. No tensor was defective in isolation: each was correctly laid out and correctly paired. The decisive step was to calibrate probes on a healthy Apple model and then to compare *reader–reader* with *reader–writer* relations. In AFM, query–key and gate–up relations were as healthy as in CodeLM, but value–output (copy heads) and up–down (FFN neuron coupling) relations were weak. In a trained network, the sign of the coupling between up-row i and down-column i agrees with the sign of the gain that scales the write. In AFM the *number* of negatively coupled channels equalled the number of negative entries of g_2 in every layer—while their *positions* disagreed (Figure 7, left). Matching each channel's 11-layer sign signature against the table gave a constant offset of one element. Read aligned, the agreement rises to 98–99.9% (right) and the model works: 66% top-1 against Apple's outputs immediately, rising to 87.5% with the llama.cpp implementation and exact tokenization.

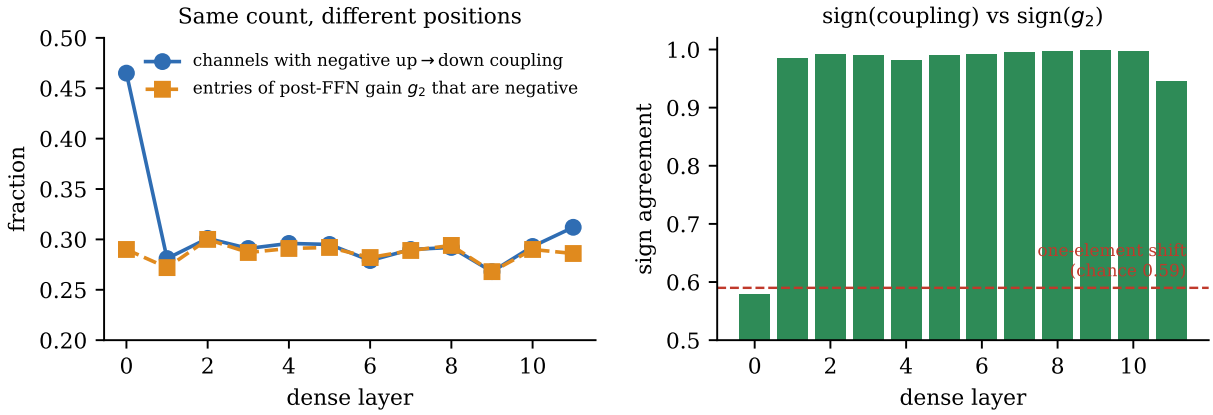


Figure 7. Left: per layer, the fraction of residual channels with negative up→down coupling (from the weights) and the fraction of negative entries in the decoded post-FFN gain g_2—equal counts. Right: per-channel sign agreement after reading the table aligned; a one-element shift gives chance level. Layer 0 is the exception because its input is the raw embedding.

7. Validation instruments and their calibration

The gain-table defect was found with diagnostic probes whose readings on a known-good model were established first. This section describes the reference, the calibration model and the probes; Section 11 gives the evaluation protocol proper.

The reference. `afm -t 0` (a small Swift wrapper around the Foundation Models API) produces Apple's greedy answers. At temperature zero the token Apple emits at each answer position is its most likely one, so teacher-forced top-1 agreement compares our prediction with an absolute target. The API exposes no probabilities.

The copy probes. Two probes are used below, in the sense of the circuits analysis of [17]. The *OV copy probe* asks whether an attention head, when it attends to a token, writes that token back to the output. For a head with value and output projections W_V, W_O and a token t with embedding e_t, it computes the readout $E\,W_O W_V\,e_t$ over the vocabulary and records the rank of t itself (0 is

best). A head's score is the median rank over a fixed sample of tokens, and a layer's score is that of its best head. A head whose output is unrelated to token identity scores near half the vocabulary size (131,072 for AFM), which we mark as chance. The *same-token QK probe* is the analogous rank for attention scores: among keys built from sampled tokens, the rank of the key that belongs to the query's own token.

A healthy reference: CodeLM v3. Xcode's on-device code model ships as a GPU MPSGraph package. Its MLIR (via `-getIR`) is a complete, readable tamm transformer—pre-norm, standard residual, RoPE then QK-norm, SwiGLU, KV reuse from layer 34, int4 weights with group-32 scales—with the same backbone shape as Apple's other 3B-class models (56 layers, 35 KV-owning, 2048 wide, 16/2 heads). We ran it on the GPU from our own process through `MPSGraphExecutable` (100% induction accuracy) and reproduced its logits in NumPy exactly (correlation 1.00000, 100% argmax agreement). Its tokenizer and embeddings are unrelated to v7-3b's, so it cannot donate weights, but it calibrates the probes.

What the calibration showed.

- *Zero-centred literal gains are normal* in tamm (CodeLM's pre-attention gain: mean +0.003, 46% negative).
- *The embedding–weight "alignment ratio" used earlier in this work is not diagnostic*: a healthy model scores the same (1.0–1.4).
- *The OV copy probe is diagnostic*: CodeLM has several heads that map a token to itself at rank 0–16, even after naive 2-bit rounding; decoded AFM had none below rank 1000 (Figure 8). Every alternative V/O layout scored at chance, which argued against a layout error and directed attention to the gains.
- *The same-token QK probe* found AFM's query–key circuits healthy (rank-0 heads) and confirmed the head pairing $h \mapsto \lfloor h/2 \rfloor$.

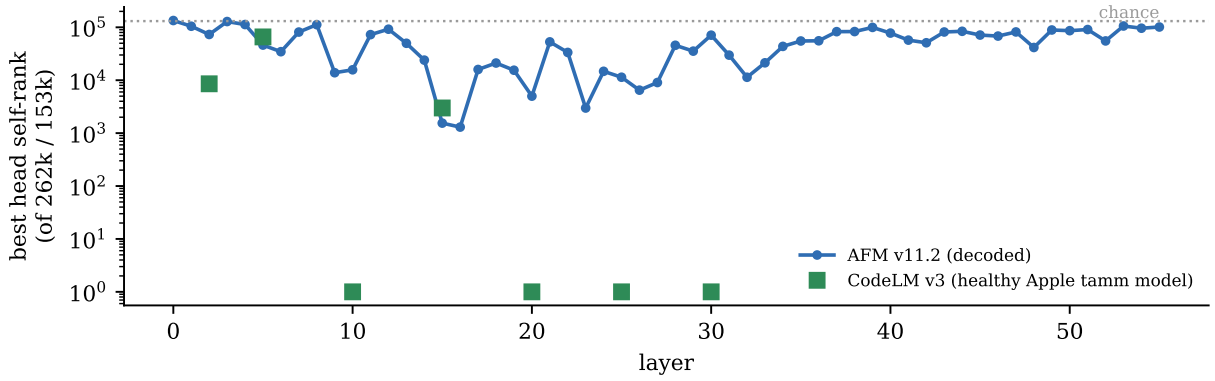


Figure 8. OV copy probe: for each layer, the best attention head's median rank of the input token in $E\,W_O W_V\,e_t$ (0 is best; the dotted line marks chance, half the vocabulary). All 56 AFM layers and seven sampled CodeLM layers are shown. Healthy CodeLM has rank-0 heads; AFM's raw $W_O W_V$ peaks at weak copying because in AFM the write passes through the signed gain g_0, whose positions were scrambled by the table shift.

Apple's int8 calibration as a ground truth. The ANE program stores per-layer scales for its int8 K/V cache. Apple presumably chose them from real activations, in which case they bound the true magnitudes. With the corrected gains, the reconstruction's values stay inside Apple's ranges, and its keys stay inside up to the 99.9th percentile; only the most extreme 0.1% of keys would clip

(Figure 9). This is an independent check of the internal scales. (Figure 9 was produced before the layer-0 corrections of Section 12; with the correct layer-0 input the layer-0 values use only part of Apple's range, so Apple's layer-0 scale is not a tight bound.)

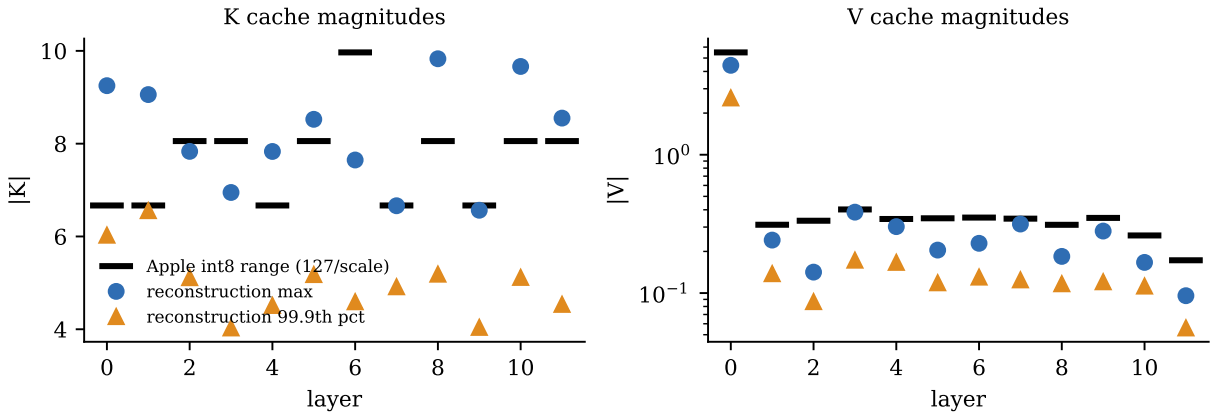


Figure 9. Reconstructed key and value magnitudes (dense layers, one prompt) against Apple's calibrated int8 ranges. Layer 0's values are large because its input is the un-normalised embedding.

8. Apple's runtime: the prompt template and the expert schedule

Two things the model files do not contain decide what the model sees and which experts it uses: the prompt template (the metadata only names it, `afm_cider_v2`) and the rule for choosing experts. Both live in code, in two private frameworks of the dyld shared cache: `TokenGenerationCore` renders and tokenizes prompts, `TokenGenerationInference` drives the model. `dyld_info -disassemble` prints both with Swift symbol names. One detail matters for reading them: Swift stores string literals of up to 15 bytes inside the instruction stream (`mov`/`movk` immediates), so `<start_of_turn>` and the other markers do not appear in a strings dump.

8.1. The prompt template.

`PromptPreprocessingTemplateAfmCiderV2Convention` takes its special-token table from the older Cider v1 convention and overrides only audio and reasoning entries. Decoding the immediates gives turn start `<start_of_turn>`, turn end `<end_of_turn>`, and roles `system\n`, `developer\n`, `user\n`, `model\n`; the "thought budget: none" and "developer static instructions" entries are empty. Its `cacheableSystemPrefix` concatenates turnStart, the system role with Apple's default instructions, the thought budget, turnEnd, a newline, then turnStart and the developer role. A FoundationModels conversation therefore renders as

<bos><start_of_turn>system\n*default*<end_of_turn>\n
<start_of_turn>developer\n*instructions*\n<end_of_turn>\n
<start_of_turn>user\n*prompt*<end_of_turn>\n<start_of_turn>model\n

where *default* is the `systemInstructions.default` text of the adapter metadata ("A conversation between a user and a helpful assistant. You are a foundation model developed by Apple. . . .") and the developer turn holds the app's instructions. The public API confirms the rendering: `SystemLanguageModel.tokenCount(for:)` accepts transcript entries and returns the length of the rendered prompt, which equals the length the inference service logs ("asked to run 71 tokens") for the same conversation. Table 9 compares it with our rendering. A template that folds everything into one system turn and omits the instructions passed by the reference client scores markedly lower; with the exact template, teacher-forced agreement rose from 87.5% to 90.5% and the first answer token from 30 to 38 of 40.

Transcript (I = instructions, P = prompt, R = response)	Apple `tokenCount`	our template
P only	16	16
I (empty), P	62	62
I only	55	55
I, P	71	71
I, P, R, P	86	86

Table 9. Prompt lengths. A session without instructions still carries the system turn and an empty developer turn (62 tokens, seen in the live log); each instruction segment ends with a newline; earlier answers are closed with `<end_of_turn>\n`.

8.2. **The expert schedule.** The inference log of a single request already shows the shape of the algorithm: a "prompt expert compute function" (`lora_48_ifp3_r48_prompt_opt_dense_only_256_64`, the dense layers only) runs over all prompt tokens, then "Picking IFP experts at position=70 isPromptPick=true" for a 71-token prompt, then the sparse layers run over the *whole* prompt again with the picked experts. The dense-only function returns a `1536×64` fp16 tensor per 64-token chunk, one column per token. In `TokenGenerationInference`, `computePromptExpertState` builds the router input with `IFPDenseLayerOutput.makeExpertsTensor(tokenIndex:)`, whose loop copies the 1536 values of a single column into a 1536×1 tensor—no mean, no accumulation. The router (`project_experts`, the network of Figure 4) then yields the top-23 experts per sparse layer. During generation, `repickExpertsAtBoundary` calls the same picker, and `IFPExpertPicker` rejects any intermediate pick whose position is not a multiple of `expert_selection_frequency` (an `sdiv`/`msub` remainder test against 32). With prompt length P, token t therefore uses the experts chosen from x_{12} at

$$s(t) = \max(P - 1,\ 32\lfloor t/32 \rfloor),$$

so every prompt token uses the last prompt token's choice. Positions are zero-based and absolute: they count every token of the rendered prompt, including `<bos>` and the template's markers, and continue through the answer. For the 71-token prompt of the log above, the prompt occupies positions 0–70 and all of it uses the pick made from position 70. The generated tokens at positions 71–95 keep that pick. The token at position 96 is the first multiple of 32 after the prompt: the router is run on that token's own layer-12 input and its pick applies to positions 96–127; the next re-pick is at position 128.

Averaging over the prompt instead scores much lower (79.3% teacher-forced), and the single-token rule outperforms per-token routing (Section 12).

An outside check of the router. Each pick also logs `CacheExpertExpertCache: ... replaced=`n, the number of (layer, expert) choices that changed. We ran six long generations on Apple's model, collected its 47 re-picks, replayed the same token sequences through our model and counted how many of our choices change at the same positions. With the single-token rule on the layer-12 attention input the two series agree closely (Figure 10: mean 26.7 versus 27.1 experts, correlation 0.84); prefix means correlate at 0.21–0.53 and the FFN-input variant changes three times too many. This supports the router input, the router network and the rule independently of any answer text. It is an indirect test: the log reports how many (layer, expert) choices changed, not which, so agreement in the counts does not establish that the selected experts are identical. The correlation is a Pearson coefficient over the 47 re-picks; Section 13 repeats the comparison on 135 re-picks.

9. The llama.cpp implementation

The patch to llama.cpp [6] adds one architecture, `afm`, in `src/models/afm.cpp`, together with the supporting changes listed below.

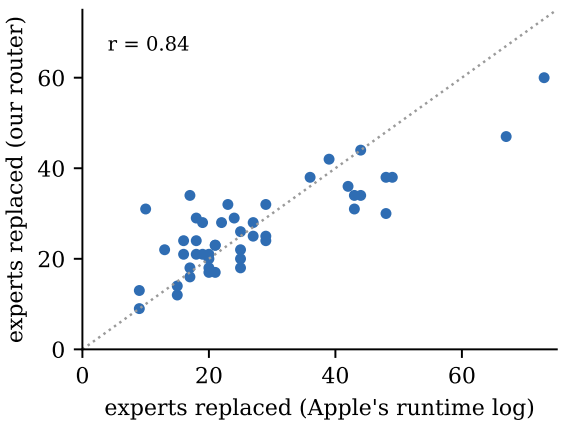


Figure 10. Experts replaced at each re-pick during generation: Apple's runtime log versus our router replaying the same tokens. Each point is one re-pick (47 re-picks in six generations); r is the Pearson correlation.

- **Graph.** Layer 0 takes the dequantised embedding row as is (no multiplier) and, for generated tokens other than re-pick tokens, adds the input bias (Section 12); each later layer normalises the previous output with g_3; queries and keys use `rms_norm`, NEOX RoPE, then a gain multiply; the residuals follow Figure 2. Sparse layers run the always-on experts as a "shared expert" SwiGLU of width 5888 and the routed experts with `mul_mat_id`, summed by one row reduction with no weights. The router network lives on layer 12 and its logits are sliced per layer. The output applies the double norm and the tied embedding.
- **KV reuse.** Layers 35–55 call attention with null K/V, and the cache maps them to layer 34. This needed a two-line fix in the non-sliding-window attention path, which assumed K/V were always present.
- **Metal kernels.** Metal's batched expert matmul is instantiated for specific experts-per-token counts; we added 14 and 23 (ifp2, ifp3). CUDA uses a generic path for such counts, and Vulkan supports Q2_K expert matmuls, so no change is needed there.
- **Apple's expert schedule.** The router's logits for each sequence persist in a small device buffer (64 sequence slots) owned by the model. A custom graph input computes, per micro-batch on the host, which logits every token uses: a run of several tokens of one sequence is a prompt chunk and takes its last token's logits, which are stored; a single decode token re-picks only when its position is a multiple of 32 and otherwise reads the stored choice. Because the indices are data rather than graph structure, llama.cpp's graph reuse and multi-sequence batching stay correct. Speculative verify batches are handled as single decode tokens, with two position-tagged picks per sequence so that rejected draft tokens leave no trace (Section 15). `AFM_ROUTE_TOKEN=1` restores per-token routing.
- **Embedding scale.** llama.cpp's generic `build_inp_embd()` multiplies by the GGUF's `embedding_scale` (a Granite feature), so an architecture graph that applies the scale itself applies it twice. Apple uses no multiplier (Section 12); the `afm` loader therefore ignores the key, which also keeps GGUFs written by earlier versions of the converter correct.
- **The Neural Engine SiLU.** The activation is the ANE's piecewise-linear table, built from standard ggml operations: a clamp, a magic-constant floor, two 38-entry `get_rows` tables and a relu for the right tail, then rounded to fp16. It costs about 45% of prompt speed and 20% of generation speed on Metal; `AFM_EXACT_SILU=1` switches back to the exact function.
- **Router on the CPU.** The router's top-23 margins (median 0.003) are close to the error of Metal's tiled matmul ($8 \cdot 10^{-5}$ against $2 \cdot 10^{-7}$ on the CPU; 0.002 expert swaps per layer-pick).

Its weights are therefore placed on the CPU, which makes picks identical across backends. The routing state and SiLU tables stay on the layer's device. Agreement is unchanged within noise.

- **int8 KV emulation.** The GGUF carries Apple's per-layer K/V scales; keys and values are rounded and saturated to int8 before caching. Rounding uses the fp32 magic constant $1.5 \cdot 2^{23}$, because `ggml_round` misbehaved on Metal.

Speed. Table 10 gives throughput on the test machine. The figures depend on the backend, the batch shape and the load on the machine; they were not collected under a controlled protocol and are indicative only.

Machine	Apple M4 (10 cores), 24 GB, macOS 27.0 (26A428)
Build	upstream llama.cpp at the base commit in `llama.cpp-patch/BASE_COMMIT` plus the patch; CMake Release, Metal
To reproduce	`llama-bench -m afm-v11.2-ifp.gguf -p 512 -n 128 -r 5`
Prompt processing	about 150 tokens/s
Generation	12–21 tokens/s (range over runs under different machine load)

Table 10. Throughput of the default configuration (ifp3 budget, Neural Engine SiLU table).

The patch applies to upstream llama.cpp at the base commit recorded in the repository; the repository's workflow builds Windows (CPU, Vulkan, CUDA), Linux (CPU, Vulkan) and macOS (Metal) binaries from it.

10. The converter

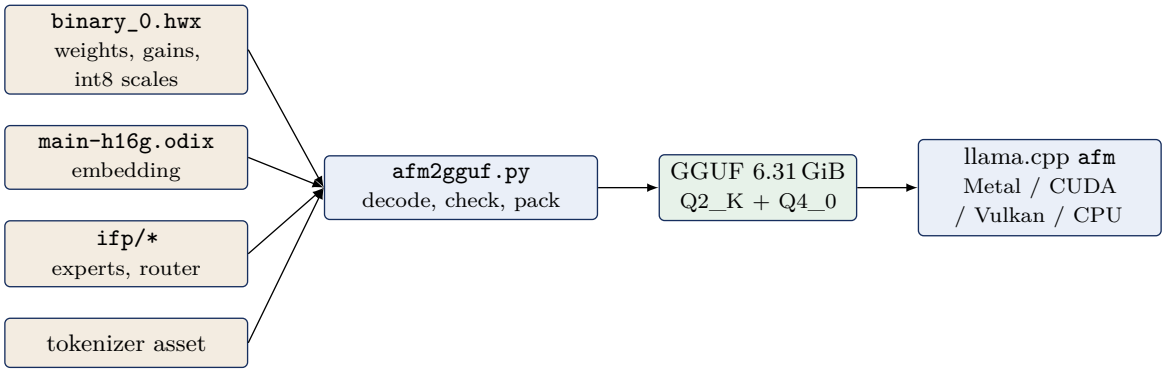


Figure 11. Conversion runs on the Mac that holds the assets (2–3 minutes); the GGUF then runs anywhere.

`afm2gguf.py` needs only NumPy and gguf-py and contains no Apple data. It finds assets by their `AssetSpecifier`, decodes everything directly, and refuses to write if a check fails: palette and container layout, 3504 units in 219 chunk-sets, the router mask, the tokenizer layout, the embedding scale array (chosen by row-norm consistency) and—decisively—the gain-table alignment, chosen by the FFN coupling-sign agreement (0.99 for the true start, 0.59 for the one-vector-early candidate that passes every shape test). Its output is byte-identical, tensor by tensor, to the export used for the measurements in this report. With `--draft`, `--vision`, `--pico`, `--pico-vision` and `--pico-adapter` it writes the draft model, the image encoder, the 300M base model, its image encoder and its adapters instead; for the image encoder, every gain is taken from the program's own constant fetches; the 300M model's RoPE base is fitted from the table in its own asset (500 000).

11. Evaluation methodology

Reference. The reference is Apple's own model, queried through the public Foundation Models API with greedy sampling (temperature 0) and the instructions "You are a helpful, concise assistant.". Each prompt is sent in a new session by a small command-line client (`tools/afm.swift`; `tools/eval/collect_oracle.sh`). The API returns text only: no token ids, probabilities or internal state. The answers are deterministic; repeated requests return the same text. The development references were collected on macOS 27.0 build 26A5378n. The held-out references were collected on 14 November 2026 under build 26A428; five development prompts re-collected on that day returned byte-identical answers, so we treat the model as unchanged.

Prompt sets. Three sets are used (Table 11); all three are published with the code. All prompts are short English requests of the kind an assistant receives: factual questions, explanations, lists, short writing tasks. The *first development set* (40 prompts) was written at the start of the project, and every debugging step described in this paper used it. The *development extension* (190 prompts) was added on 8 November 2026 because the first set moves by about ±0.4% from noise alone. It was then used to choose between candidate implementations (Table 15), so it is development data as well. Nine of its prompts repeat prompts of the first set verbatim; the combined set therefore has 230 entries and 221 distinct prompts, and we keep the duplicates so that totals match across tables. The development sets were not built from a category scheme and carry no category labels.

The *held-out set* (100 prompts) was written on 14 November 2026, after the converter and the runtime were final. It has no prompt in common with the development sets, and covers eight categories: 20 factual questions, 20 explanations, 10 short creative writing tasks, 10 requests for practical advice, 10 lists, 10 arithmetic and word problems, 10 language tasks (translation, grammar, summarising) and 10 questions on programming and computing. Apple's answers were collected once and the evaluation was run once. No prompt was removed or replaced after any result had been seen, and no implementation choice was made on this set.

Set	prompts	answer tokens	tokens per answer, median (range)	role
Development, first set	40	1,127	21 (2–90)	all debugging and ablations
Development, extension	190	5,445	22 (2–147)	choice between candidate variants
Held-out	100	2,900	20 (3–320)	final evaluation, run once

Table 11. Prompt sets. Token counts include the end-of-turn token of each answer.

Metrics. *Teacher-forced top-1 agreement* is the primary measure. The rendered prompt and Apple's answer are fed to the reconstruction, and at each answer position we ask whether the reconstruction's most likely next token is the token Apple produced. Because Apple decodes greedily, that token is Apple's own most likely token, so the measure compares the two models' top choices given an identical context. It is not a measure of free-running output and not a measure of task accuracy. *First-token agreement* is the same test at the first answer position only. *Free-running identity* lets the reconstruction generate its own greedy answer (up to 300 tokens) and counts the answers that equal Apple's character for character after surrounding whitespace is removed. A single different early token changes the rest of an answer, so this measure is far stricter and is dominated by the first few tokens.

Token alignment. Since the API returns text, Apple's token sequence is not observed. We remove leading and trailing whitespace from Apple's answer, tokenise it with the reconstructed tokenizer without interpreting special tokens, and append the end-of-turn token. The scored positions are

these tokens: the first is predicted from the last token of the rendered prompt and the end-of-turn token from the last answer token. Prompt tokens and role markers are not scored. This assumes that Apple's model emitted the canonical tokenisation of its answer. The assumption cannot be checked directly; the tokenizer reproduces Apple's token counts on 81 of 81 test texts (Section 12). No reference answer in any set was empty, truncated by an error, or a guardrail refusal.

Unit of analysis and uncertainty. Tokens of one answer are correlated: they share a prompt, a routing decision and a context. Treating the answer tokens as independent trials would understate the uncertainty, so the unit of resampling is the prompt. Confidence intervals are 95% percentile intervals from 10,000 bootstrap resamples of prompts [19], with token-level agreement recomputed on each resample. We also report the distribution over prompts (median agreement and the number of prompts on which every token agrees) and absolute counts.

Development and held-out results. Several implementation choices were made by agreement with Apple on the development sets: the prompt template, the layer-0 input, the routing schedule, the Neural Engine SiLU table and the treatment of the input bias at re-pick tokens. Each also has independent support in Apple's code or program, but scores on prompts that guided those choices can overstate agreement on unseen prompts. The held-out set exists to measure that, and Section 12 reports it first.

12. Fidelity

This section reports agreement with Apple's model under the protocol of Section 11: first the held-out result, then the development sets and the ablations that were run on them.

Held-out result. On the 100 held-out prompts the default configuration reaches **95.66%** teacher-forced top-1 agreement (2,774 of 2,900 tokens; 95% interval 94.8–96.5% by prompt-level bootstrap). The first answer token is Apple's for 91 of 100 prompts, and 51 of 100 free-running answers are identical to Apple's. Table 12 sets this beside the development sets. The held-out agreement lies inside the interval of the combined development set and slightly above its point estimate, so we find no sign that the development figures were inflated by the choices made on them. The share of identical free-running answers is higher on the held-out set, which contains more questions with short factual answers.

Set	prompts	tokens	teacher-forced top-1	95% interval (bootstrap)	median prompt agreement	prompts at 100%
Held-out	100	2,900	**95.66%**	94.8–96.5%	97.9%	49 of 100
Development, first set	40	1,127	96.98%	96.1–97.8%	98.3%	18 of 40
Development, extension	190	5,445	94.82%	94.2–95.5%	95.9%	72 of 190
Development, combined	230	6,572	95.19%	94.6–95.7%	96.8%	90 of 230

Table 12. Teacher-forced top-1 agreement of the default configuration with Apple's model, by prompt set. Intervals resample prompts, not tokens. "Prompts at 100%" counts prompts on which every answer token is predicted. All rows are from one run of the released evaluation script on 14 November 2026. On the combined development set that run differs by 5 of 6,572 tokens from the earlier run reported in Table 15 (95.27%); differences of this size occur between runs on different backends and builds (Section 13).

Development sets and ablations. The remaining results in this section were obtained on the development prompts and were used to choose between implementation variants; they show which details matter, and their absolute values should be read with that in mind.

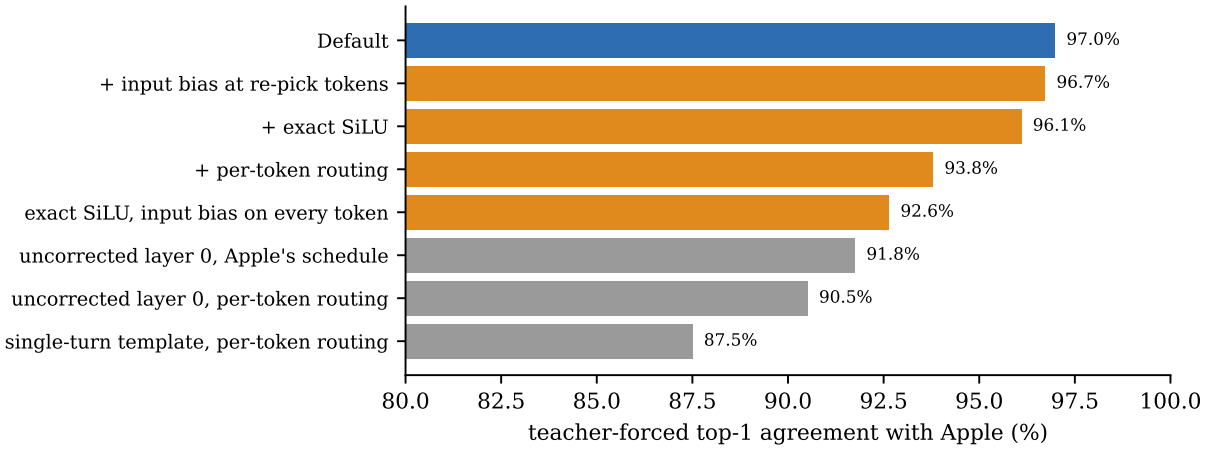


Figure 12. Teacher-forced top-1 agreement with Apple's greedy answers on the first development set (40 prompts, 1127 tokens). The horizontal axis starts at 80%, so bar lengths exaggerate the relative differences. Blue: the default configuration (Apple's schedule, the Neural Engine SiLU table, no input bias at re-pick tokens); the three rows below it each change one further property. "Uncorrected layer 0" denotes an embedding scaled by 100 and the input bias on every token. The exact template is worth three points, the layer-0 corrections (no embedding multiplier, input bias on generated tokens only) four more, and Apple's routing schedule outperforms per-token routing.

Configuration (exact template)	teacher-forced	first token	identical answers
Default	**96.98%**	38/40	17/40
+ input bias also at re-pick tokens	96.72%	38/40	17/40
+ exact SiLU	96.10%	37/40	17/40
+ per-token routing	93.79%	36/40	12/40
Uncorrected layer 0, Apple's schedule	91.75%	35/40	7/40
Uncorrected layer 0, per-token routing	90.51%	38/40	11/40

Table 13. Configurations on the first development set. The default uses Apple's schedule, the Neural Engine SiLU table and no input bias at re-pick tokens. The rows above the rule are *cumulative*: each indented row keeps the changes of the rows above it and adds one more, so the fourth row differs from the default in three properties. The two rows below the rule are complete configurations; "uncorrected layer 0" denotes an embedding scaled by 100 and the input bias on every token. "Identical answers" = our free-running greedy answer equals Apple's exactly (up to 300 tokens); one different early token sends the rest of an answer elsewhere, so this measure is dominated by the first few tokens.

Ablation (10-prompt set, single-turn template)	top-1
always-on + 23 routed experts	85.6%
always-on experts only	62.9%
routed experts only	1.4%
16 instead of 23 routed / 14 routed	83.5% / 82.1%
standard (pre-norm) residual instead of x/y	0%

Table 14. Ablations on a 10-prompt set, with the earlier single-turn template. Each row is an independent change from the first row. Both expert groups are needed, the routed count matters, and the residual form is essential.

The layer-0 input: no multiplier, and a bias only for generated tokens. An initial analysis fitted an input multiplier k and found the score flat for $k = 8$–14. That scan was invalid: llama.cpp's generic embedding code already applied the GGUF's `embedding_scale`, and the AFM graph applied it again, so the scan really covered $k = 80$–140. Dumping llama.cpp's intermediate tensors and comparing them with an independent NumPy forward revealed a factor of exactly 10 at the layer-0 input. Scanning the true multiplier with the expert picks held fixed gives a sharp optimum at $k = 1$ (mean log-loss of Apple's tokens 0.140, against 0.143 at 0.75 and 1.25 and 0.240 at 100). This agrees with Apple's program: the embedding gather has no multiply.

With $k = 1$ the body of the network and the router disagreed. The body wanted the input bias added, but the router's picks were best when the bias was negligible. This was established by computing the picks from a forward at a larger k, or with the bias removed; a version that changed only the layer-0 attention balance did not help (78%). Apple's ANE program has two procedure families. The prompt procedures (`prompt_opt`) add a zero constant at layer 0. The extend procedures, which compute generated tokens, add the bias vector. Applying the bias to generated tokens only is one consistent model for both parts. It reaches 95.9% with the picks held fixed and 96.1% in the llama.cpp runtime, which gives the bias to single-token decode steps and not to prompt chunks. Flipping the bias's sign costs two points, so the vector is genuine; it belongs to one procedure family only.

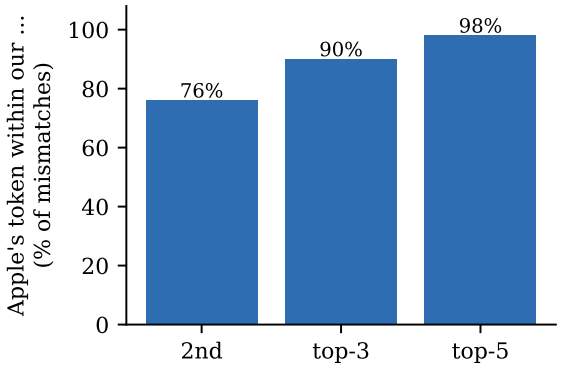


Figure 13. Rank of Apple's token at the 93 disagreements of the uncorrected-layer-0 configuration: Apple's token is our second choice in 76%, in our top three in 90% and in our top five in 98%; the median logit gap to it is 1.15, and 45% of the misses are within one logit.

The tokenizer. Apple's `SystemLanguageModel.tokenCount` gives Apple's own token count. A tokenizer with byte-fallback tokens labelled as normal text and control tokens mislabelled matches it on only 73 of 81 texts. The converter's tokenizer matches on 81/81 (and on 16/16 texts built from

tabs, runs of spaces and indented code) and raises fidelity from 86.0% to 87.5%. Apple's tokenizer file marks all user-defined pieces with one type code; mapped naively, llama.cpp prints space runs as the escape character U+2581 instead of spaces and drops tab runs entirely, so the converter types space runs as normal text and tab runs as user-defined.

13. The remaining gap

The prompt template, the expert schedule, the layer-0 input and the activation function follow Apple's code and program. On the first development set 37 of 1127 answer tokens still differ, and on the combined development set 316 of 6572; Apple's token is almost always our second or third choice. We regard the arithmetic of Apple's compiled program as the leading explanation of this residual, and this section sets out the evidence for and the limits of that view. Attributing a residual gap to numerical noise requires care: before the layer-0 corrections a gap of eight points could be, and was, mistaken for a noise floor. The following were therefore measured directly:

(1) **The ANE's matrix multiplies are accurate.** We ran our own CoreML programs on the Neural Engine, confirming the device with Core ML's compute plan. Sums accumulate at high precision and only the fp16 output is rounded (relative error 2.8×10^{-4}, the same as fp32 accumulation with an fp16 result). The low-precision accumulation one first sees comes from Core ML's fp16 CPU path, not the ANE.

(2) **Its SiLU is a 33-knot piecewise-linear table** (knots every 0.5 on $[-8, 8]$, fitted values, for example 0.2964 at $x = 0.5$ where SiLU is 0.3112). The same 128-byte table appears 2113 times in AFM's ANE program, once per FFN task, and it is the program's only activation table. We tabulated it for all 65 536 fp16 inputs. It is exactly linear interpolation between the knots, with a left tail clamped to zero below -10.2 and $x - 0.0039$ above 8, rounded to fp16. With the uncorrected layer-0 input, emulating it is neutral (10 fixed, 8 broken) because the layer-0 error dominates. On the corrected model it fixes 14 and breaks 5 tokens (96.10% $\rightarrow$ 96.72%), so it is the default. Its softmax, exp and rsqrt are also approximate (0.3–0.4% relative error).

(3) **Apple's speculative decoding is exact.** Every FM-API adapter sets `draft_soft_match_tolerance` to 0 and turns rejection sampling off, so the draft model never changes a greedy token. The FM-API adapter is the declared-empty adapter (signature = SHA-256 of the empty string), and there is no repetition penalty.

(4) **The router is robust to arithmetic.** Computing it with fp16 accumulation moves 0.01 of 23 experts per layer-pick, and random noise strong enough to swap a third of an expert per layer changes 2% of predictions without moving the agreement.

Every weight, gain, bias, the router, the tokenizer, the template and the routing rule come from Apple's files and code; nothing was trained or fine-tuned.

Path noise. On the corrected model, the CPU and Metal backends (identical mathematics) disagree on 0.71% of answer tokens (8/1127), and their agreement with Apple is identical (96.72%). The remaining 37 misses are near-ties: Apple's token is our second choice in 84% and always in our top three, and the median logit gap is 0.40. Rounding every matmul output to fp16, as the ANE stores it, gives 96.89% (within noise). With the uncorrected layer-0 input, mathematically identical variants disagree on 0.1–2.4% of next-token choices: fp16-rounded activations against fp32, llama.cpp's RoPE kernel against an elementwise one, or one batch against a prompt batch followed by single steps. The 2-bit weights and the int8 KV cache amplify rounding, so a percent or so of residual disagreement is expected even from an exact model.

A larger test. The 40-prompt set moves by about $\pm 0.4\%$ from noise alone, so we added 190 prompts (6572 answer tokens in all) and re-ran every candidate (Table 15). The Neural Engine SiLU is worth a full point. Every other ANE detail we could emulate is neutral: fp16 matmul outputs,

int8 KV on or off, keys quantised before their gain, and fp16 RoPE tables. Truncating instead of rounding the int8 cache costs a point, so Apple rounds to nearest. Of the 316 misses of the reference row, 2.9% of tokens are missed under any routing and 1.9% only under Apple's schedule. The latter is consistent with our prompt-level pick differing from Apple's by a boundary expert or two: the 23rd and 24th router logits are a median 0.003 apart, about the size of a rounding difference in the layer-12 state. Apple's log also confirms the re-pick timing. Around a boundary the runtime steps one token at a time, and the pick at position 96 uses token 96 itself. Apple's fp16 logits never tie at the top, and its softmax errors (about 0.2% in the denominator) cancel in the post-attention RMS norm.

Variant (230 prompts, 6572 answer tokens)	top-1	fixed / broken
Default (Apple's schedule, ANE SiLU, int8 KV, no input bias at re-pick tokens)	**95.27%**	12 / 7
Input bias also at re-pick tokens (reference row)	95.19%	–
exact SiLU	94.17%	72 / 139
per-token routing	93.34%	–
fp16 rounding of every matmul and attention output	95.13%	30 / 34
int8 KV emulation off	95.07%	46 / 54
int8 rounding by truncation	94.05%	64 / 139
keys quantised before their gain	95.09%	43 / 50
RoPE cos/sin rounded to fp16	94.99%	21 / 34
re-picks that favour already-loaded experts ($\beta = 5 \cdot 10^{-4}$)	95.15%	–

Table 15. Candidates on the combined development set. Each row below the reference row is an *independent* change of one property of the reference row, not a cumulative one; the default (first row) is the reference row without the input bias at re-pick tokens. Fixed / broken = tokens that become right / wrong relative to the reference row.

Running the dense layers on the Neural Engine. As a direct test of the arithmetic, we rebuilt layers 0–11 as a Core ML program, which placed every operation on the ANE: int8 KV round and clip, the ANE's own SiLU and softmax. We fed its layer-12 state to llama.cpp for routing and the remaining 44 layers. The state differs from our emulation by about 15% (relative norm) after 12 layers; in isolation each layer's FFN agrees with our SiLU-table model to fp16 rounding (0.5%), and int8 steps in the very coarse layer-0 values amplify the differences. Agreement with Apple stays the same: 94.92% against 95.19%, but 224 of 6572 tokens flip (103 fixed, 121 broken). The control, the same program in fp32 on the CPU with exact SiLU, gives 94.42%. So two plausible versions of the hardware arithmetic disagree on about 3.4% of tokens without either being closer to Apple, which is the size of the remaining gap. Core ML compiles our graph differently from Apple's own ANE program, so even the real hardware does not reproduce Apple's rounding.

Interpretation. These experiments make compiled-program arithmetic a plausible dominant contributor to the remaining disagreement: equally defensible versions of the arithmetic differ from one another by as much as the reconstruction differs from Apple, and the misses are near-ties. They do not exclude a smaller contribution from routing or execution details that cannot be observed. Two such candidates are a prompt-level expert pick that differs from Apple's by a boundary expert (about 1.9% of tokens are missed only under Apple's schedule, see above) and behaviour of Apple's runtime between requests that our single-request tests do not exercise. Distinguishing these would require either Apple's per-layer expert lists, which the log redacts, or an execution of Apple's own compiled program on chosen inputs, which the operating system does not permit to third-party processes.

Apple's own expert decisions. Apple's runtime logs every pick per layer ("layer N experts: ..."). The expert list is marked *sensitive*, so it stays redacted even with a logging profile that

enables private data for the inference service. That profile does reveal the subsystem's ordinary private values. The cache statistics are public, though. At each pick the runtime logs `reordered`, `replaced` and `loaded`, and within one answer the cache holds exactly the current pick, so `replaced` at a re-pick counts the experts that changed. Over the 135 re-picks in the 230-prompt set, our router's predicted churn tracks Apple's with $r = 0.934$ and a mean miss of 4.6 of about 40 changed experts. This indicates that our picks differ from Apple's by at most a few of the 1012 per pick, with the caveat that the log counts changes and does not identify experts. Moving the re-pick to a neighbouring token makes the match clearly worse (5.3–5.4), which supports the schedule. Every arithmetic variant scores the same within noise: 4.50 for the ANE-computed layer-12 state, 4.60 for the default, 4.60 for fp16 matmul outputs, 4.65 for fp32 with exact SiLU, 4.67 for exact SiLU, and 4.45 without the input bias at re-pick tokens. Apple's re-picks change slightly fewer experts than ours (by 1.6 on average). A small preference for already-loaded experts fits the counts only marginally better (4.60 to 4.40) and lowers agreement (95.15%), so we reject it. Apple's code runs a dense-only function at each re-pick token, the same procedure family as the prompt, which adds no layer-0 bias. Dropping the bias at those tokens raises agreement from 95.19% to 95.27% (12 fixed, 7 broken) and is part of the default. Between prompts the counts do not track the previous prompt's pick (Apple 501 on average, our prediction 111), so the cache is not simply carried over between requests.

Further numerical and schedule experiments, run on an earlier configuration, are collected in Appendix B; they agree with the findings above.

14. Apple Intelligence feature adapters

Sections 14–17 cover the components that ship alongside the main model. Each is decoded by the same methods, but the available validation differs from one to the next, as Table 4 records.

Apple Intelligence features ship as rank-48 LoRA adapters [12] for this backbone: 26 on the test machine, covering summaries, proofreading, mail replies, smart replies, translation, message actions, reminders, Image Playground suggestions, Photos memories, dictation and others. Each asset holds `lora_i`N`_r48_48_constant_data.bin`, where N is the IFP level the feature runs at. Its metadata holds Apple's prompts for the feature (system instructions and content templates, in the `afm_cider_v2` format) and the same backbone signature as the base model. All of them carry the empty-input SHA-256 as their "adapter signature", so that field says nothing about whether an adapter is trained. The FM API's own adapter is untrained (B RMS $3 \cdot 10^{-5}$), while the feature adapters are trained (B RMS $2 \cdot 10^{-3}$ to $3 \cdot 10^{-2}$).

Layout. A file holds 218 (A, B) pairs as fp16: A as $[48, \text{in}]$, then B as $[\text{out}, 48]$. The pairs come in the order of the package manifest, which sorts the tensor names as strings: the dense segment (o, q, k, v, `linear_0`, `linear_1`, down) $\times 12$, then kv-reuse (o, q) $\times 21$, then standard sparse (o, q, k, v) $\times 23$, with the layers inside each segment in the order 0, 10, 11, 1, 2, ... rather than numerically. `linear_0` is the gate and `linear_1` the up projection, and the adapter is applied at scale 0.5. All three points match the 300M adapters (Section 17).

Checked against Apple. Apple's public Translation framework is served on device by the `machine_translation` adapter at ifp1 (the model manager's log names the asset, and the request produces speculative-decoding log lines), and its output is deterministic. The prompt is not in the adapter metadata. We recovered it from the `TranslationInference` framework and confirmed it with the token counts Apple logs for each request (51, 61, 63 and 65 tokens for four probes, all reproduced): the system turn is *"Conversation between a user and a friendly assistant. You are an expert translator."* and the user turn is *"Translate from S to T.\nS phrase:* text*\n\nT phrase:"*, with language names such as "American English" and "Chinese, Traditional".

On 200 translations of 40 English sentences into German, French, Spanish, Japanese and Traditional Chinese, teacher-forced on Apple's output, our adapter picks Apple's token at 3,380 of 3,424 positions (98.7%); the base model at ifp1 without the adapter reaches 89.2%. Table 16 gives the figures by target language; the adapter's contribution is largest for Japanese and Chinese, where the base model alone is weakest. Free-running in llama.cpp, 166 of the 200 translations are identical to Apple's. Apple translates each sentence of a multi-sentence input separately, which our test did not do. The reference is essential. An alternative decode (numeric layer order, scale 1, `linear_0` = up) passes a likelihood test on a small set of self-constructed proofreading pairs, yet scores 0.7% against Apple's translations: a decode of this kind helps slightly at a small scale and diverges at full scale. Writing Tools' Proofread action cannot serve as a reference, because it is served by a Private Cloud Compute model.

Target language	Tokens	Reconstructed adapter	Base model alone
German	725	98.8%	93.0%
French	777	99.5%	94.2%
Spanish	659	99.5%	95.8%
Japanese	659	98.2%	84.4%
Chinese, Traditional	604	97.4%	76.3%
All	3,424	98.7%	89.2%

Table 16. Agreement with Apple's Translation framework by target language: share of Apple's output tokens that the model ranks first, teacher-forced, for 40 English sentences per language, at ifp1.

A second adapter: notification urgency. Apple's suggestion service classifies every app notification with the `urgency_classification` adapter (ifp3) and logs its verdict ("Notification urgency: true"). A small test application that posts local notifications therefore provides a reference for a second adapter at a different IFP level. The adapter is a one-token classifier: the notification is urgent when the probability of "Yes" reaches 0.3771 (the adapter metadata's threshold for English notifications). The system turn is the metadata's instruction followed by "The user's locale is en_US.", and the user turn is *"[Notification]\nTitle: ...\nCommunicationFlag: False\nBody: ...\n\nIs this urgent?"*, read from `SummarizationKit`. Apple's logged token counts match this prompt exactly when the text mentions a time of day and are two tokens longer otherwise, a difference that remains unexplained. On 55 everyday notifications our adapter gives Apple's verdict for 53, the base model for 48 and the alternative decode for 46. On 30 further notifications, selected because these three variants disagree, the adapter agrees with Apple on 28, the base model on 9 and the alternative decode on 7; on the 18 of them whose prompt length matches exactly, the adapter agrees on all 18. The remaining 24 adapters share the file format and the decode, but belong to features that only Apple's own entitled processes can run. They are therefore decoded but not validated: the two checked adapters show that the decode is correct for adapters at the smallest and the largest budget, not that each of the other 24 behaves as Apple's does.

IFP levels. ifp1 and ifp2 use the first 5 and 14 always-on experts plus as many routed ones. In llama.cpp these are views of the shared FFN; at 256-aligned offsets they are whole Q2_K blocks. The level is read from the active adapter (`adapter.afm.ifp`). The level matters: run at ifp2 or ifp3 instead of its own ifp1, the translation adapter agrees with Apple less often.

15. The speculative-decoding draft model

Apple pairs the model with a small draft model for speculative decoding [13]: asset `fm_api_generic.draft`, `model_config` `v11-ifp-9m`. We separate two questions that carry different

certainty: what the asset encodes (the draft model's weights and graph, read from its program) and what Apple's runtime does with it (the speculation procedure, inferred from logs). Its metadata asks for 7 draft tokens per step, exact matching (`draft_soft_match_tolerance` 0) and no adaptive length. The asset has no weights file: everything is inside its Neural Engine program (`binary_0.hwx`, 22 procedures: 3 readouts of 8/16/64 tokens and 19 `extend` variants) and its `program.odix`.

Shape	12 layers, $d = 256$, 8 Q / 4 KV heads $\times$ 32, SwiGLU FFN 768; 9.4 M parameters plus the int4 embeddings
Layer	pre-norm: $h = \mathrm{rms}(x)g_1$; $q, k =$ RoPE (half-split, base $5 \cdot 10^5$) *then* per-head RMSNorm and gain; $x \mathrel{+}= W_o\,\mathrm{attn}$; $x \mathrel{+}= \mathrm{FFN}(\mathrm{rms}(x)g_2)$
Weights	4-bit palettes: each projection is 16 units (one per ANE core), each a sorted 16-entry fp16 codebook, then input-major nibbles; no row scales. Gate tiles carry the 128-byte ANE SiLU table
Vocabulary	100,000 of the 262,144 tokens (a sorted id list in `program.odix`); 95.9% of Apple's answer tokens are in it
Embeddings	int4 codes for the 162,144 other tokens, then the 100,000 draft tokens, in the base model's 8-token interleave, times one per-dimension fp16 scale s; no per-token scale
Readout	tied: the readout procedure's constant tile is $[\,0 \mid s\,]$, i.e. $h \cdot s + 0$, then the int4 matmul over the draft vocabulary

Table 17. The draft model as recovered from its Neural Engine program.

15.1. **What the asset encodes.**

Reading the dataflow. The 132 weight convolutions (11 per layer) are the tasks with Neural Engine opcode `0x46` and a kernel offset. Their roles follow from the surrounding tasks. In layers 1–11 the first 128-channel projection is written straight to the cache by a tile-destination DMA, so it is V. The second passes through RoPE, a 32-wide RMS reduction and a gain multiply before its cache write, so it is K. Layer 0 orders them q, k, v. The attention score scale $1/\sqrt{32}$ sits in the QK matmul's output-scale register, and the softmax is an exp2 with a $\log_2 e$ prescale. The 768-wide FFN hidden is laid out by L2 address: the three up and three gate tiles are written to descending addresses, so the hidden is $[\mathrm{tile}_2 \mid \mathrm{tile}_1 \mid \mathrm{tile}_0]$. In the last layer the compiler re-banks these buffers and leaves one destination register implicit, so we use the conv order there. Norm gains come from `__TEXT,__const`, one 0x4000-byte block per procedure, addressed by the tile-source offsets. In layer 2 the query gain multiply reuses the tile the key gain loaded; the compiler had merged the two. The query and key gains of a layer agree up to sign flips inside rotation pairs and a few ulps, so the norm cannot be moved before RoPE. The final norm computes $\mathrm{rsqrt}(\mathrm{mean}((0.1x)^2) + 1.19 \cdot 10^{-7}) \cdot 0.1$, an fp16 overflow guard equivalent to $\epsilon = 1.19 \cdot 10^{-5}$.

Validation. A NumPy forward of this description predicts Apple's own next answer token (from the 3B model, teacher-forced) 42.95% of the time on the 40 prompts (1127 tokens) and 37.7% on the 230 (6572 tokens). Every alternative tested collapses: cyclic instead of unit-blocked output rows 0.0%, reversed RoPE direction 16.7%, query head $h \to$ KV head $h \bmod 4$ 6.7%, K and V swapped 1.5%, FFN tiles in conv order 2.3%, raw codes without s 0.3%. The input scale is a sharp optimum at 1 (0.25: 27.6%, 4: 36.5%). The ANE SiLU table changes nothing (42.95% either way). Simulating a 7-token chain with exact match, the draft's proposals are accepted 0.61 tokens per step on the 230 prompts, so each target call yields 1.61 tokens.

15.2. **What Apple's runtime appears to do.**

The evidence. Apple's draft program does not run from our process, but Apple's runtime reports what it did. After every output token the inference service logs running totals (`log stream`, subsystem `com.apple.tokengenerationinference`, category `SpeculativeDecode`): `countTotalAcceptedDraftTokens`, `countTotalRejectedDraftTokens`, `speculatedComputedCount`. A rise in the first at output token n means Apple's draft had proposed token n; a rise in the second means it had not. That is one bit about Apple's draft per decision, at a known position in a token sequence we also have. Read as a greedy chain the bits did not fit: Apple accepted 53% of the decisions where our draft's top token was right only 40% of the time, and the two agreed on 78%. The counters never speculate deeper than two tokens, the draft's programs take 8 tokens, the metadata sets `draft_steps` to 7 and `draft_use_maximum_likelihood_tree`, and when Apple accepted, the token was among our draft's five best in 97.5% of the cases with a sharp edge after the fifth.

Competing hypotheses. We considered three accounts of the procedure. (a) A greedy chain of 7 draft tokens, the usual scheme: rejected by the figures above (acceptance 53% against 40%, agreement 78%). (b) A maximum-likelihood tree, as the metadata flag suggests, with 7 nodes. (c) The same with 6 or 8 nodes. Hypothesis (b) fits best: each round scores the last real token and 7 draft nodes, the 7 most likely nodes of a depth-2 tree by joint probability, and a token is accepted if it is a node at its level. Replaying that tree with our draft on Apple's answers (60 prompts, 1435 logged decisions, 1331 of them more than one token away from a program switch, see below) gives the same decision as Apple's draft in 95.6% of the cases, with the same acceptance rate (55.7% against 55.9%); a tree of 6 or 8 nodes misses the acceptance rate (53.6%, 57.3%), and an unrelated model with that rate would agree 51% of the time. The remaining disagreements are near-ties: where the token's joint probability is within a factor $2^{1/4}$ of the tree's threshold the two agree 60% of the time, beyond a factor 2 they agree on 99.2% (1134 decisions), beyond a factor 4 on 99.7% (951). The GGUF run by llama.cpp gives the same greedy decisions as the NumPy forward on all 1435.

Limits of the identification. The log yields one bit per decision. The tree is the simplest procedure we found that is consistent with the metadata, the depth of speculation and the acceptance rate, and under it our draft reproduces Apple's decisions wherever they are not near-ties. That is strong evidence that the decoded draft model is correct, since a wrong model would not reproduce the decisions under any procedure we tried. It is weaker evidence for the exact tree construction: other procedures with the same acceptance set at depth 2 would be indistinguishable from it in this log.

Two details of Apple's runtime are visible as well. When the sequence reaches 96 or 128 tokens the runtime moves to the next larger fixed-context program, and that round is logged as rejected whatever the draft proposed (36 such rejections of tokens our tree contains); the 104 decisions within one token of those lengths are therefore left out. And the gain of Apple's scheme is larger than a chain's: 0.56 of the tree's decisions are accepted.

15.3. **In llama.cpp.** `afm2gguf.py --draft` writes the draft as architecture `afm-draft` (61 MiB): the codes as Q4_0 with $d = 1$ (exact), s as `token_embd_scale` and `output_scale`, F16 projections (palette values are fp16), and an `output.bias` of $-\infty$ outside the draft vocabulary. The bias matters: the best draft logit is negative at about 1% of positions, so zero rows would sometimes win. On Metal, its logits match the NumPy reference to 0.03.

Speculative decoding places a requirement on the target graph. A verify batch [last token, 7 draft tokens] is several tokens of one sequence, which would otherwise be treated as a prompt chunk: one shared expert pick, persisted, and no input bias. Verify batches are therefore recognised (every token requests logits and the run does not start at position 0) and processed exactly as single decode tokens: input bias except at re-pick positions, and routing on the latest re-pick token at or before each token. Draft tokens can be rejected after they have re-picked, so the routing state keeps two

picks per sequence, each tagged with its source position. A pick from a position the batch starts at or after belongs to rolled-back tokens and is ignored. With this handling, 8-token verify batches reproduce one-token-at-a-time decoding to $2 \cdot 10^{-5}$ in the logits.

Lossless, but not faster in llama.cpp. With greedy sampling, 7 draft tokens and no confidence cutoff (Apple's settings), `llama-server` with the draft returns exactly the plain answers on all 40 prompts, on Metal and on the CPU, and the plain answers are unchanged by the patch. About 11% of drafted tokens are accepted, as the simulation predicts. Speculation is nevertheless slower in llama.cpp. Measured back to back, it was 2.1× slower on Metal and 3.5× slower on the CPU (1.8× with 3 draft tokens). A verify batch costs several single decodes here, because the 2-bit expert matmuls are compute-bound and every draft step has a fixed per-call cost. Apple's Neural Engine runs fixed-shape 8-token `extend` programs, so a verify step costs about as much as one token, and 0.6 extra tokens per step is a real gain there. `tools/eval/speculative.py` reproduces these measurements.

16. The image encoder

Images reach the model through a separate asset, `instruct_3b.image_encoder` (`model_cls_name ImageTokenizer`). It holds one Neural Engine program with six procedures, one per patch count $N \in \{1152, 1728, 2304, 3600, 4608, 5760\}$, and `swift_preprocess_weights.bin`. The program's inputs are the patches $[N, 3, 14, 14]$, an additive position embedding $[N, 1024]$, RoPE rotation matrices $[N, 32, 2, 2]$ and an additive key mask. Its output is $N/9$ embeddings of width 1536, which the 3B model reads between `<start_of_image>` and `<end_of_image>`. Apple's runtime log names the sizing rule: the aspect ratio is kept, the image is scaled to at most 338 688 pixels (1728 patches), and both sides are snapped to multiples of the 12-patch attention window.

Shape	ViT [16], 24 layers, $d = 1024$, 16 heads × 64, SwiGLU 2816; 358 M parameters with the merger
Block	$h = \mathrm{rms}(x)g_1$; q, k, v with bias; RoPE then per-head RMSNorm and gain on q, k; attention inside 12×12-patch windows, full attention in layers 7, 15, 23; $x \mathrel{+}= W_o a + b_o$; $x \mathrel{+}= \mathrm{SwiGLU}(\mathrm{rms}(x)g_2)$
Positions	learned 16 × 16 table (bicubic-resized to the patch grid) added after the patch embedding, then a pre-norm; 2D RoPE on interleaved pairs: pair $2f$ rotates by the row, pair $2f{+}1$ by the column, both at $\theta_f = 10000^{-f/16}$
Token order	window-major; inside a window the 3×3 merge groups in raster order; inside a group the patches in raster order
Merger	final RMSNorm; 9 consecutive tokens concatenated (9216) → 4096 → LayerNorm ($\epsilon = 10^{-6}$) → GELU → 2048 → 1536, no biases
Weights	int4 (16-entry LUT × fp16 row scale) except the patch embedding and the last projection (fp16)

Table 18. The image encoder as recovered from its Neural Engine program.

Evaluation data. The image tests in this section use 13 images: seven photographs between 640 × 480 and 4096 × 2160 pixels, and six synthetic 672 × 504 or 504 × 672 images (a coloured square, circle and triangle, the word "HELLO", an arithmetic expression and coloured stripes). Apple's reference caption for each is its greedy answer through the public API, which accepts an image attachment, with the instructions used throughout. Per-token cosine similarities compare our encoder's output with the output of Apple's encoder for the same pre-processed input, token by token. Thirteen images make a check, not a benchmark, and we give no uncertainty estimates for them.

Reading the weights. The int4 blocks resemble the 3B model's, but the header length depends on the convolution's flags: a 16-entry fp16 LUT, 32 zero bytes, the row scales, and the row biases *only if* the convolution has a bias (Neural Engine register 1, bit 0x10). A 128-byte activation table precedes all of this when the convolution applies an activation (bit 0x20000). A fixed 128-byte header is correct only for 16-row blocks with a bias; under that assumption every bias-free block (all FFN and merger weights) is read 4 to 12 inputs out of place, with its last inputs taken from padding. The block sizes look the same either way because each unit is padded to 64 bytes. Compiling known weights with Apple's compiler settles the question: a 9216→128 projection, a bias-free up projection and a SiLU-gated projection decode exactly with the flag rule and at chance level (6%) with the fixed header.

The effective bias is the stored bias times the row scale, and times 2^{e-32} when the convolution's task writes the Neural Engine bias register `NEBias` with exponent field e (its value field is 0). In this encoder only the V convolutions of layers 18 and 19 do so ($e = 31$); missing it left their biases twice too large, which costs 0.001–0.007 of agreement with Apple's encoder. Projection tiles are written to descending addresses, so channel order is the reversed tile order. The weights are split over two kernel sections; each block is assigned to the section where its LUT is valid. That rule is unambiguous for 1293 of 1297 blocks, and the remaining 4 follow their neighbours.

Norm gains are [bias | scale] constant tiles that the planar engine fetches; we take every gain from its fetch offset rather than from a layout formula, because the global-attention layers use a different constant layout. The compiler also folded a 0.01 pre-norm scale into the encoder: the last residual add scales its residual operand by 0.01 (a planar-engine operand scale, the same register that holds 1/8 in the global layers), and 0.1 is folded into each of the last layer's up and down projections, which are ten times smaller than every other layer's. The converter undoes this. Activations are 33-knot piecewise-linear tables: SiLU on $[-8, 8]$, GELU on $[-4, 4]$, with explicit linear tails.

Ground truth from Apple's own encoder. Unlike the language model, the encoder's MPSGraph executable runs on the Neural Engine from our own process (about 0.8 s per call), so Apple's encoder is a direct oracle for any input we build. A copied package does not load ("Program load failed"), so weights cannot be patched to bisect. We therefore used constructed inputs. When all N tokens are identical, every attention layer returns V, so the encoder becomes a per-token function of one 1024-vector; and the merger can be inverted from the output. With the fixed header the inversion stalls at cosine 0.991 for every seed; with the flag rule it reaches 0.99999. The NumPy forward then matches Apple's output at cosine 0.987 on constant-token inputs and 0.95–0.97 per token on real images. One further correction is invisible to constant-token inputs: layer 0 emits its key projection before its query projection. With q and k of layer 0 exchanged, the match on real images rises to 0.98–0.99 per token. The llama.cpp projector requires two further measures to follow: Apple's activation tables in place of the exact SiLU and GELU (with exact activations it matches Apple's encoder at 0.975), and an antialiased bicubic resize, since a plain 4-tap bicubic aliases when a large photo is reduced (0.84 against Apple's encoder for a 4096×2160 photo, 0.985 with antialiasing). From the image files the projector matches Apple's encoder at 0.98–0.99, its Metal and CPU paths agree to 0.9999, and 6 of 13 free-running captions are identical to Apple's.

Every wiring choice that can be varied has a sharp optimum: the alternative V/O and gate/up/down pairings score 0.16–0.23, and in every layer each other choice of V group is worse; on a real image, swapped q/k scores 0.09, half-split RoPE pairs 0.50, x-before-y 0.51 and transposed patches 0.85. Fitting per-layer branch scales gains less than 0.4 points, so the remaining percent is not a wiring error. A Core ML replica of our forward run on the Neural Engine was less faithful still (0.76), so the numerics of Apple's compiled program are the most likely source of the remaining difference.

Through the 3B model: the RoPE layout. Our prompt places the question, a newline and the image in the user turn, 266 tokens in total, exactly the count Apple's runtime logs for the same

request. Several things the encoder program cannot tell us are chosen by Apple's host code: the pixel normalisation, the position table's interpolation, and how the 32 RoPE rotations of a head are split between the row and the column. Comparing against Apple's encoder cannot settle these, because both encoders receive whatever inputs we build. The 3B model's behaviour can, indirectly (Table 19). With the obvious layout (row on pairs 0–15, column on 16–31) the language model received colours and scene gist but no geometry. Shuffling the 192 image tokens hardly changed anything, and asked whether a red circle is left or right, top or bottom, it answered "Left" and "Top" for all four pictures, where Apple's model answers all four correctly.

The correct layout *alternates*: pair $2f$ rotates by the row and pair $2f{+}1$ by the column, both at frequency θ_f. With it all four spatial answers are right with margins of 5–12 logits. The mean log-probability of the key words of Apple's captions (circle, HELLO, equation, Golden, balloons, ...) rises from -2.99 to -0.20. The other host choices are then sharp: a negated rotation mirrors the image ("HELLO" at -8.2), bilinear instead of bicubic position interpolation is worse (-0.26), and so is a transposed position table (-0.21). ImageNet normalisation is slightly better than CLIP's (-0.20 vs -0.22). With CLIP's statistics a plain white image is barely "white" (0.2-logit margin over gray, against 6.5 with ImageNet's), and CLIP's mean vector does not occur anywhere in the operating system's shared cache.

Step	Recovered setting	Evidence	Rejected alternative
Image size	aspect ratio kept; at most 338,688 pixels; sides snapped to multiples of 12 patches	Apple's runtime log	—
Resize filter	antialiased bicubic	Apple's encoder: 0.985 against 0.84 on a 4096×2160 photo	plain 4-tap bicubic
Pixel normalisation	ImageNet mean and deviation	3B model: key-word log-probability -0.20 against -0.22; white-image margin 6.5 against 0.2 logits	CLIP statistics
Position table	16×16, bicubic interpolation	3B model: -0.20 against -0.26 and -0.21	bilinear; transposed table
RoPE layout	alternating pairs: $2f$ row, $2f{+}1$ column	3B model: 4 of 4 spatial answers; key-word log-probability -0.20 against -2.99	row on pairs 0–15, column on 16–31; negated rotation
Token order	window-major; 3×3 groups in raster order	program structure; Apple's encoder	other window, group and patch orders
Activations	33-knot piecewise-linear SiLU and GELU	table bytes; Apple's encoder: 0.975 with exact functions	exact SiLU and GELU

Table 19. Image pre-processing and host-side choices: the recovered setting, the evidence for it and the alternatives that were rejected. "3B model" denotes the indirect test through the language model's answers.

Teacher-forced on Apple's captions of 13 images (168 tokens), our encoder gives the 3B GGUF Apple's next token 94.0% of the time (NLL 0.29), Apple's own encoder on the same inputs 92.9% (NLL 0.29), and no image 58.9% (NLL 2.11); these figures were measured before the layer-0 correction above. Free-running through llama.cpp with Apple's instructions, 6 of the 13 captions are identical to Apple's character for character, among them "The word "HELLO" is written in black capital letters on a white background.", and the rest are close paraphrases ("The image shows a math problem: "2 + 3 = ?"." for "...a mathematical equation: 2 + 3 = ?.").

In llama.cpp. `afm2gguf.py --vision` writes the encoder and merger as a multimodal projector (685 MiB, F16), projector type `afm` in llama.cpp's `mtmd` library. Tokens are processed in Apple's order, with a window mask on 21 of the 24 layers. The query and key head dimensions are permuted so that llama.cpp's vision RoPE (rotating dimensions j and $j{+}32$, row for $j < 16$, column above)

reproduces Apple's alternating interleaved pairs. Its embeddings match our NumPy forward at cosine 0.990. On an M4 an image encodes in 0.66 s on Metal.

17. The 300M base model

Apple ships a second, smaller language model, display version `afmplus-v11.0-pico` (asset `com.apple.fm.language.instruct_300m.base.generic`). Fifteen other assets on the same Mac carry the `instruct_300m` name, among them the safety guard (`instruct_300m.safety`), `misc_safety`, `mm_guard`, `action_validator` and several classifiers. In a memory capture of Apple's runtime, the 300M model's input buffer also held a guard turn ending in the model answer `'safe'`. The asset holds one Neural Engine program with 22 procedures (11 with rank-32 adapter slots, 11 with rank-64) and `program.odix` with the embedding.

Shape	24 dense layers, $d = 1024$, 16 query / 4 KV heads $\times$ 64 (query head h reads KV head $\lfloor h/4 \rfloor$), SwiGLU 3200; 299 M parameters in the 24 layers (24×12.45 M), not counting the $262\,144 \times 1024$ embedding (268 M int4 entries), which is shared with the readout
Block	the draft model's layer (Section 15): $h = \mathrm{rms}(x)g_1$; RoPE then per-head RMSNorm and gain on q, k; attention scale $1/8$; $x \mathrel{+}= W_o a$; $x \mathrel{+}= \mathrm{SwiGLU}(\mathrm{rms}(x)g_2)$
Positions	RoPE base 500 000, half-split; the cos/sin table ships in `program.odix` directly after the embedding
Embedding	int4 codes $\times$ one fp16 scale per dimension; readout $\mathrm{rms}(x)\, g_{\mathrm{out}}\, s$ against the same codes, plus a mask of -40000 on 144 unused ids
Weights	int4: 16-entry fp16 LUT $\times$ fp16 row scale; 960 blocks (40 per layer)

Table 20. The 300M base model as recovered from its Neural Engine program.

Decoding. The model is read as the draft model and the image encoder are: from the Neural Engine task stream rather than from symbol names. Five properties of the format are essential, several of them shared with the image encoder; a decode that misses them recovers every tensor yet produces no text. (1) The int4 header is 96 bytes (LUT, 32 zero bytes, 16 row scales; the bias flag is clear), followed by the codes and 32 bytes of zero padding; an assumed 128-byte header shifts every input axis by 4 positions (12 for the 128-row blocks). (2) The norm gains are not folded away: the planar engine fetches all 97 of them (four per layer and a final one) from `__TEXT,__const`, beside a $[0 \mid s]$ tile that scales the hidden state before the readout and the -40000 vocabulary mask. (3) The embedding is a plain 8-token interleave starting at the data section; an apparent lane "skew" and 4-row shift are artefacts of a wrong base address. (4) The per-layer multiplier (9.5–12.0) applied with the key gain is not a temperature. It is the int8 scale of the key cache, and the $q \cdot k$ matmul's own fp16 scale register divides it back out: their product is 0.1250 in all 24 layers. Values are likewise stored in int8 (scales 305 down to 10.5) and rescaled in the attention-times-V matmul. (5) The two 256-wide projections are emitted in different orders: layer 0 computes K first, every later layer V first, as in the draft model. We assign them by dataflow: K is the one whose output passes through RoPE and the head norm. Block order within a tensor follows the destination addresses (the query blocks and the up projection's are written to descending addresses), as for the encoder.

Results. Apple exposes this model through no public API, so its outputs cannot be compared as for the 3B model. We evaluate it as a language model instead. On two passages of plain English (172 and 102 tokens) its perplexity is 38.2 and 24.6, the median rank of the correct next token is 2, and it ranks it first 29% and 39% of the time. On random token sequences repeated twice it copies the second half 78% of the time, the signature of trained induction heads [18]. Each of these properties is necessary: with a positional K/V assignment induction is 0% and perplexity above 100 000; swapping K and V in layer 0 as well leaves perplexity above 20 000; halving or doubling

the logits raises it threefold to ninefold, so the readout is calibrated as decoded. Greedy decoding through llama.cpp:

> The capital of France is → *Paris.* Water boils at → *100°C.*
> Once upon a time, there was a little girl named → *Lily. Lily was a little girl who loved to play with her toys...*
> (chat turn) What is 12 plus 7? → *12 plus 7 is 19.* Write one sentence about the ocean. → *The ocean is a vast, mysterious expanse of water that stretches for miles, teeming with life, diverse ecosystems, and endless horizons.*

In llama.cpp. The layer is the draft model's, so `afm2gguf.py --pico` writes it with the `afm-draft` architecture and no new runtime code: the embedding as Q4_0 with unit scales (exact), the projections as F16, the full 262 144-token vocabulary with Apple's mask as the output bias (721 MiB). Its greedy continuations are identical to our NumPy forward's.

Adapters and the safety guard. Every 300M adapter, including the default one inside the base package (`lora_32/64_constant_data.bin`), is a flat file of fp16 (A, B) pairs, A as $[r, \text{in}]$ and B as $[\text{out}, r]$, per layer in the order attention output, q, k, v, `linear_0`, `linear_1`, FFN output. The layer order requires care. The pairs follow the package manifest's tensor names sorted as strings, so the layers come as 0, 10–19, 1, 20–23, 2–9. Read in numeric order, every adapter looks inert (the guard below never says "safe"); with a plain numeric-string sort (0, 1, 10, ...) the adapters work only partially. `linear_0` is the gate (log-probability of the right first token −0.10 against −0.55 as up), the Neural Engine program scales the product by 0.5, and every role contributes.

The default adapter. With the correct order, the base package's own rank-64 adapter is a trained default: it lowers the base model's perplexity from 38.2 to 30.8 and from 24.6 to 21.4 (llama.cpp, 128-token windows: 26.9 to 22.1). Twelve task adapters of rank 32 or 64 ship on the test machine, and all share its starting point: their weights correlate 0.33–0.99 with the default adapter of the same rank, where unrelated adapters would correlate near 0.

The safety guard. The text safety guard (`instruct_300m.safety`, rank 64) carries Apple's instruction: answer `safe`, or one line per problem of the form `category,mention` or `category,action` over 14 categories (hate_harassment, violence_gore, self_harm, jailbreak, vulgarity, ...). Run through llama.cpp with that instruction as the system turn, it answers `safe` for every benign prompt and answer tested, and labels harmful text in the requested format: `self_harm,action` for a plan to self-harm, `jailbreak,action` for "Ignore all previous instructions...", `vulgarity,action` for swearing. It separates talking about a thing from doing it: a history of the battle of Verdun gets `violence_gore,mention`, a friend's past suicidal thoughts `self_harm,mention`, a news report of an arrest `safe`. The smaller `mm_guard` (rank 32, nine categories) agrees on everything except the swearing, which it calls `safe`; vulgarity is not one of its categories. `afm2gguf.py --pico-adapter NAME` writes any of them as a llama.cpp LoRA with Apple's instructions in its metadata, and `tools/afm300m_classify.py` runs a classifier as Apple renders it.

Checked against Apple's guardrail. Apple's public FoundationModels API runs the same guard: it rejects a request with "May contain unsafe content" when the guard's labels map to `unsafe` under the API's row of Apple's policy table (`feature_output_category_mapping.json`: eight labels such as `violence_gore,action` or `self_harm,action` block; mentions, jailbreak attempts and vulgarity do not). The comparison is behavioural: the API reports a decision, not the guard's labels or scores. On 26 prompts (benign questions, mentions, nine harmful requests, two jailbreaks, two vulgar ones) our guard with that table predicts Apple's decision on all 26, including the cases our guard labels but Apple's policy lets through. On a second set of 74 prompts (32 of which Apple stops) it predicts 69; the five misses are borderline (four insults or indirect threats that ours labels as an action and Apple lets through, and a hobby question about lock picking that Apple stops), so 95 of 100 in all.

One request Apple stopped at its output check ("Response may contain sensitive…"), which our guard already flags on the input.

The image encoder. The 300M model has its own image encoder, `instruct_300m.image_tokenizer` (`ImageTokenizer`, model id `afm-plus-v11-300m-instruct-4bit-pal`). Its program has the 3B encoder's structure exactly: 24 layers, windows of 12×12 patches with full attention in layers 7, 15 and 23 (counted from the attention matmuls per layer), the same 3×3 merger, and a last projection to 1024 instead of 1536. Its position table differs, so it was trained separately.

Three details differ from the 3B encoder. The weight palettes are sorted real-valued codebooks rather than integers; there is no folded 0.01 scale (the last layer's weights are in line with the others); and 15 of its 96 attention bias tensors come from tasks that write the `NEBias` exponent ($e = 28$–31), so their biases are 2^{e-32} times the stored value. Read without that factor they are 2–16 times too large, which no weight statistic reveals but Apple's own encoder does: on identical inputs the agreement is cosine 0.45–0.54 per token without the factor, and 0.61–0.78 on photos and 0.97 on constant inputs with it. Two further corrections, both checked against Apple's encoder, bring it to 0.68–0.87: layer 0 emits its key projection before its query projection (as in the 3B encoder), and the five final-norm gains that are fp16 subnormals are taken as zero.

The merger normalisation. The remaining discrepancy is a single layout property of the merger. Its normalisation is a depthwise convolution whose kernel holds a (bias, gain) pair per channel in 16 units of 256 pairs, and the units are *interleaved* over the channels: channel c is entry $\lfloor c/16 \rfloor$ of unit $c \bmod 16$. Reading the units one after another puts every gain and bias on the wrong channel. Nothing in the bytes indicates this, and the 3B encoder is nearly insensitive to it (0.992 to 0.994 on constant inputs), because its gains and biases are nearly uniform; the 300M encoder's are not. It was identified with constant-token inputs, where Apple's encoder is a per-token function: the reconstruction's output was uniformly 17% too small in norm, the 3B encoder's was not, and refitting only the 4096 biases of that layer on 54 inputs raised held-out agreement from 0.956 to 0.997, and on photos (never fitted) from 0.68–0.87 to 0.95–0.97. The fitted biases correlated with the stored ones only under the transposed unit order. With that order and no fitted quantity the 300M encoder agrees with Apple's at 0.998 on constant inputs and 0.990–0.999 per token on photos (the 3B encoder: 0.989–0.997), and llama.cpp's projector at 0.98–0.99 from the image files.

The two programs were also compared register by register: they are identical task for task except for the bias exponents, the folded 0.01, and the output width, and every weight block and bias tensor of one correlates with its counterpart in the other ($r \approx 0.98$ for weights, slope 1.00 for all 96 bias tensors under the exponent rule). The same rule explains every one of the 96 tensors' power-of-two ratio to the 3B encoder's biases. The same decoder therefore covers both (`afm2gguf.py --pico-vision`); the 3B projector it writes is unaffected.

Image classifiers. In llama.cpp the encoder runs as the existing `afm` projector. The only runtime requirement is that the `afm-draft` graph apply the per-dimension embedding scale to text tokens only, so that image embeddings enter unscaled. With the default adapter the 300M model describes images ("The image contains the text "HELLO" in a stylized font."; "a blue square with a white background"). The image classifiers are adapters of the same kind as the guard: `misc_safety` holds Apple's instructions for violence/gore classification and for counting people ("0, 1, 2"), and `prepubescent_safety` checks for children shown with adult features. On stock photos from Keynote's themes the people counter answers 0 for all four photos without people, 1 for all five single portraits (without the bias rule it misses a person whose face is hidden by a hood), 2 for two women, and 2 for three boys, where 2 is the largest answer allowed. The two safety classifiers answer `safe` for every benign image tested, including synthetic red paint splatter; they were not tested on harmful images, so nothing can be concluded from this about their safety performance.

The following were excluded as causes of the residual difference between the two encoders: the Neural Engine's softmax (its 33-knot 2^x and $2/m$ tables and fp16 $\log_2 e$), fp16 saturation in the norms, reduced-precision palettes, and the kernel-section choice of the three blocks that are valid in both sections; none changes the agreement. Apple's encoder is also numerically stable: perturbing an input by 10^{-3} moves its output by less than 0.001 in cosine.

18. Limitations

What the primary measure does and does not show. Teacher-forced agreement with the public API measures how often two models share a top choice given an identical context. It is not parameter-by-parameter execution equivalence, and it is not task accuracy. Free-running identity is much lower (Table 3) because one differing token redirects an answer.

Unobserved internals. Apple's API exposes neither probabilities nor internal activations, and the runtime log redacts the selected expert indices. The routing schedule is therefore validated through counts of changed experts, not through the identities of the experts, and the tokenisation of Apple's answers is assumed, not observed.

Numerical equivalence. The arithmetic of Apple's compiled Neural Engine program is not reproduced exactly. We regard it as the leading explanation of the remaining disagreement (Section 13), but that is a hypothesis; a contribution from unobserved routing or execution details is not excluded.

Evaluation data. The prompts are short, single-turn and English. Long contexts, multi-turn conversations, other languages (apart from the translation adapter's targets), tool calling and guided generation were not evaluated. The development prompts were used at several stages of debugging; only the held-out set is free of that. All three sets were written for this study, in a similar style, and are small by the standards of model evaluation.

One machine, one model version. All results come from one M4 Mac, macOS 27.0, and one version of each asset. Apple updates its assets independently of the operating system; the formats, offsets and scheduling rules reported here may change with any update.

Companion components. Only two of the 26 feature adapters have a reference. The 300M base model has none and is assessed only as a language model. The safety guard is compared with the public API's decisions, not with the guard's internal output, on 100 prompts written for the purpose; the image safety classifiers were run on benign images only. The image tests use 13 images. None of these supports a general statement about safety performance or caption quality.

Lossless conversion. The equality of the converted weights with Apple's values is established by construction and by a sampled comparison of 55 million weights, not by an exhaustive one (Section 5).

19. Related work

Apple's descriptions of its models. Apple has described its foundation language models and their adapter-based deployment in a technical report [1], and the pruning method behind the sparse feed-forward network in a paper on Instruction-Following Pruning [2], in which a mask predictor reads the instruction and selects which feed-forward parameters to activate. Those publications state the principles. The present work documents one shipped instance: its dimensions, its storage, the router's input and the schedule on which the selection is refreshed, none of which they specify.

The Neural Engine. Apple has published guidance on structuring transformers for the Neural Engine [3]. Outside Apple, the hardware and its software stack have been documented by experiment [4, 5]. This paper relies on the same kind of observation, applied to one compiled program: its weight encoding, constant layout and activation tables.

Building blocks. The recovered architecture combines known components: RMS normalisation [8], rotary position embeddings [9], grouped-query attention [10], SwiGLU feed-forward blocks [11], low-rank adapters [12], a SentencePiece vocabulary [14] laid out as in Gemma 3 [15], a vision transformer [16] as image encoder, and speculative decoding [13]. The post-normalised residual of Section 4 and the cross-layer key/value reuse are, to our knowledge, not described in Apple's publications.

Runtime and diagnostics. The portable implementation builds on llama.cpp [6] and its GGUF format [7]. The diagnostic probes follow the circuit-level analysis of transformers [17, 18]: copying heads and induction behaviour serve here as tests of whether a set of decoded weights forms a working model.

We are not aware of prior public work that reconstructs this model's weights into a portable runtime and measures the result against Apple's own model.

20. Responsible use

What is published. We publish our own code: the llama.cpp architecture patch, the converter, the evaluation scripts and the prompt sets (`https://github.com/yu314-coder/afm-ifp-teardown`). The code contains no Apple data.

What users must provide. The converter reads the model assets that Apple Intelligence has installed on the user's own Mac and writes the GGUF there. Without those assets it produces nothing.

What is deliberately not distributed. We publish no weights, tokenizer data, decoded assets or converted model files, and no reference answers from Apple's model beyond the short quotations in this paper. The model's weights belong to Apple. We have asked Apple whether redistribution would be permitted.

Scope of this statement. We make no claim about how Apple's licence terms or any law apply to running the converter or to using its output; that assessment is separate from this work and rests with each user. The safety guard and the image classifiers are documented so that they can be studied, and the tests reported here do not establish their safety performance. The project is unofficial and is not affiliated with or endorsed by Apple.

21. Conclusion

A substantial part of a proprietary on-device model can be recovered from locally installed assets and observable platform behaviour. For `afmplus-v11.2-ifp` this comprises every stored tensor, the computation graph, the tokenizer, the prompt template and the expert-selection schedule, and a portable runtime that executes them on macOS, Linux and Windows. The principal architectural findings are a residual stream in which both additions carry normalised values, key/value reuse across the last 21 layers, and a sparse feed-forward network whose experts are chosen once per prompt from a single token and refreshed every 32 positions.

What was validated directly is the agreement of the resulting model with Apple's own: 95.66% of greedy next-token choices on held-out prompts under teacher forcing, with the weight layout confirmed by Apple's compiler and the prompt rendering by Apple's token counter. What remains open is exact arithmetic. The reconstruction is not numerically identical to Apple's compiled Neural

Engine program, the remaining disagreements sit at near-ties, and whether arithmetic alone accounts for them is not settled.

Future work should prioritise independent reproduction on other machines and asset versions, larger held-out evaluations that include long and multi-turn contexts, and a more exact account of the compiled program's arithmetic.

Appendix A. Negative results

The following approaches were unsuccessful. They are grouped by failure mode, and Table 21 states the diagnostic lesson of each group.

Failure mode	Examples	Diagnostic lesson
Parsing and alignment	forward variants before the gain fix; fixed 128-byte header; numeric layer order; RoPE rescaling	When every variant scores zero, the defect lies upstream of all of them. Check offsets and headers against a known input compiled by the vendor's own tools.
Runtime and routing	pooled routing rules; routing from the FFN input; an adapter as the missing part	Read the rule from the runtime's code. A rule fitted to outputs can look plausible while another component is still wrong.
Uncalibrated measures	weight statistics; residual-collapse measures	Calibrate every diagnostic on a model known to be healthy before trusting its reading.
Misread references	request log as a draft reference; between-prompt cache counts	Establish what a log line counts before using it as ground truth.
Execution access	backbone on the Neural Engine; patched or replicated encoder	The vendor's program cannot be run or modified from outside; a replica compiled by other tools is a different program.

Table 21. Negative results by failure mode.

Parsing and alignment. Errors in where and how bytes were read.

- More than ten forward variants tried before the gain alignment was corrected: Gemma-style $1+g$ gains, standard residual, swapped or negated RoPE tables, embedding scales, alternative row stripes, GQA maps, within-vector gain permutations, per-row scale permutations—all 0% top-1, because the defect was elsewhere.
- Decoding the 300M model from symbol names, compiled probes and permutation searches: these recover the tile element order exactly but never produce text. The conclusions they suggest—no norm gains ship, the embedding has a lane skew and a 4-row shift, the residual-writing projections hide an unreadable layout, some layers' FFNs "hurt"—are all consequences of the assumed 128-byte header, the wrong embedding base and the positional K/V assignment (Section 17).
- Reading the 300M adapters' layers in numeric order. Every adapter then appears inert, and the base package's default adapter appears slightly harmful, as a placeholder would. The files follow the manifest's string-sorted tensor names (Section 17).
- Before the alternating RoPE layout was identified, we attempted to restore spatial understanding with RoPE position rescaling (the RoPE table has 481 rows; $\times 10$ or fit-to-480 positions), window-relative positions, other window, group, patch and output orders, cropped position tables, collapsed LLM positions for the image tokens, bidirectional attention over them, rescaled embeddings, no int8 K/V emulation and image-before-question prompts. None helped; each leaves the axis layout unchanged and therefore could not.

Runtime and routing. Wrong assumptions about what the runtime does.

- Pooled expert routing rules (mean of inputs, of logits, of probabilities; prompt or user turn; prefix mean, vote counts): all below per-token routing, and all wrong—Apple routes from a single token (Section 8). They appear plausible only under an incorrect prompt template.
- Routing from the layer-12 FFN input instead of its attention input: 89.4% teacher-forced, and three times too many experts changed per re-pick (109 versus Apple's 36).
- A LoRA adapter as the missing component: the Foundation Models API's generic adapter for v11.2 is untrained (B RMS $3 \cdot 10^{-5}$), so the base model is already what the public API runs. Its "signature" (the SHA-256 of the empty string) proves nothing, because the trained feature adapters carry the same one (Section 14).

Uncalibrated measures. Diagnostics that do not distinguish a defective model from a healthy one.

- Weight statistics such as "alignment ratios" and residual-collapse measures: uncalibrated, and on a healthy model indistinguishable from AFM.

Misread references. Log values that do not measure what they appear to.

- Apple's `MODLM current context` N `asked to run` K `tokens` log as an oracle for draft acceptance: it interleaves the guard model, the target and the draft (and several generations) with no stream id, and the draft's catch-up calls are only ever 1 or 3 tokens, so per-step acceptance cannot be read off it. (The `SpeculativeDecode` counters can; see Section 15.)
- Reading Apple's between-prompt expert changes from the cache log: those counts (361–556 of 1012) include cache eviction while the service is idle and do not measure pick differences; only re-picks within one generation do.

Execution access. Attempts to run or alter Apple's programs directly.

- Running Apple's backbone on the ANE from our own process: the kernel refuses LLM base-model requests from non-platform binaries, even with AMFI disabled.
- Patching Apple's image encoder to bisect it: a copied MPSGraph package does not load on the Neural Engine. A Core ML replica of our encoder on the Neural Engine was less faithful (0.76) than NumPy (0.987).

Appendix B. Experiments on the uncorrected configuration

A further set of experiments was run before the layer-0 corrections (embedding $\times 100$, input bias on every token). With Apple's schedule, 74 of that configuration's 93 misses are also missed with per-token routing, so they do not depend on routing. At 89 of the 93 our choice is an ordinary text token, so Apple's output masking is not involved. Table 22 lists the numerical experiments.

Variant of the baseline (teacher-forced, 1127 tokens)	top-1
Baseline: Apple's schedule, int8 KV emulation, uncorrected layer 0	91.75%
int8 KV emulation off	90.77%
no re-pick during generation	90.95%
re-pick takes effect one token later	91.39%
activations rounded to fp16 (residual / + router input / + every sublayer output)	91.39%
RoPE angles computed in fp16 / + fp16 cos and sin	91.75% / 91.57%
attention scale $\times 0.8$ / $\times 0.9$ / $\times 1.1$ / $\times 1.25$ (of $1/\sqrt{128}$)	87.8 / 90.1 / 89.9 / 89.9%
tanh soft-cap of attention scores at 50 / 30 / 20 (Gemma-style)	91.4 / 90.4 / 89.5%
our own fp32 RoPE instead of llama.cpp's (same mathematics)	91.84%

Table 22. Numerical and schedule experiments on the uncorrected-layer-0 baseline. Emulating Apple's int8 KV cache and re-picking every 32 tokens are both worth close to a point, the attention scale peaks sharply at $1/\sqrt{128}$, and there is no attention soft-cap; storing activations or RoPE angles in fp16 changes nothing beyond the noise floor.

Appendix C. GGUF layout

Tensor	Shape (rows × cols)	Type
`token_embd`	262144×1536	Q4_0
`input_bias`, `output_norm`, `output_norm_2`	1536	F32
`blk.L.attn_q`	2048×1536	Q2_K
`blk.L.attn_k`, `attn_v` ($L \leq 34$)	1024×1536	Q2_K
`blk.L.attn_output`	1536×2048	Q2_K
`blk.L.attn_q_norm`, `attn_k_norm` ($L < 34$)	128	F32
`blk.L.{post_attention,ffn,post_ffw,layer_output}_norm`	1536	F32
`blk.L.ffn_{gate,up}` / `ffn_down` ($L < 12$)	3072×1536 / 1536×3072	Q2_K
`blk.L.ffn_{gate,up}_shexp` / `ffn_down_shexp` ($L \geq 12$)	5888×1536 / 1536×5888	Q2_K
`blk.L.ffn_{gate,up}_exps` / `ffn_down_exps`	$348 \times 256 \times 1536$ / $348 \times 1536 \times 256$	Q2_K
`blk.12.ifp_router_fc1` (+bias), `_norm`, `_fc2` (+bias)	256×1536, 256, 15312×256	F32

Table 23. 805 tensors, 6.31 GiB. Metadata keys under `afm.*`: block count 56, embedding length 1536, head counts 16/8, key length 128, expert count 348, experts used 23, shared expert width 5888, leading dense blocks 12, KV-reuse first layer 35, RoPE base 5×10^5, RMS ε, embedding scale, per-layer K/V int8 scales, and a Jinja chat template.

References

[1] T. Gunter et al. Apple Intelligence Foundation Language Models. arXiv:2407.21075, 2024.

[2] B. Hou, Q. Chen, J. Wang, G. Yin, C. Wang, N. Du, R. Pang, S. Chang and T. Lei. Instruction-Following Pruning for Large Language Models. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. arXiv:2501.02086.

[3] Apple Machine Learning Research. Deploying Transformers on the Apple Neural Engine. 2022. `https://machinelearning.apple.com/research/neural-engine-transformers`

[4] M. Hollemans. Everything we actually know about the Apple Neural Engine (ANE). `https://github.com/hollance/neural-engine`

[5] S. Bryngelson. Apple Neural Engine: Architecture, Programming, and Performance. arXiv:2606.22283, 2026.

[6] G. Gerganov and contributors. llama.cpp. `https://github.com/ggml-org/llama.cpp`

[7] GGUF file format specification. `https://github.com/ggml-org/ggml/blob/master/docs/gguf.md`

[8] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems*, 2019. arXiv:1910.07467.

[9] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.

[10] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. arXiv:2305.13245, 2023.
[11] N. Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.
[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.
[13] Y. Leviathan, M. Kalman and Y. Matias. Fast Inference from Transformers via Speculative Decoding. *Proceedings of the 40th International Conference on Machine Learning*, 2023. arXiv:2211.17192.
[14] T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. *EMNLP: System Demonstrations*, 2018.
[15] Gemma Team. Gemma 3 Technical Report. arXiv:2503.19786, 2025.
[16] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929, 2020.
[17] N. Elhage et al. A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread*, 2021.
[18] C. Olsson et al. In-context Learning and Induction Heads. arXiv:2209.11895, 2022.
[19] B. Efron. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

Email address: euler.yu@gmail.com